Table of Contents

# The Complete Apache Kafka Solution Guide

### From Fundamentals to a Production-Grade Streaming Pipeline

> A consolidated reference covering Kafka fundamentals, internals, producers, consumers, cluster/broker architecture, offset management, cloud connectivity, security, and a full end-to-end capstone project (Wikimedia real-time streaming pipeline).

---

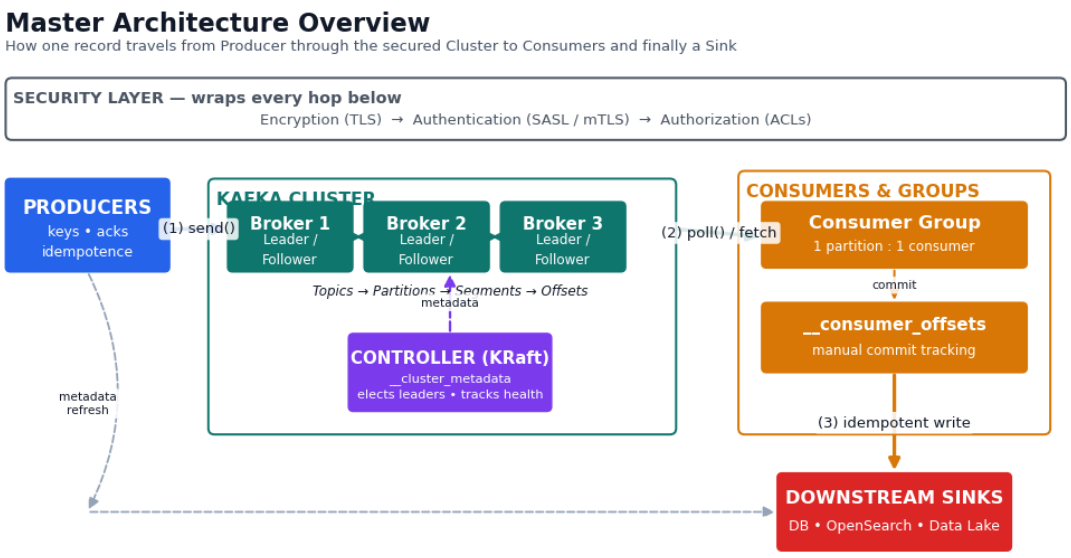
## Table of Contents


1. Master Architecture Overview
2. Chapter 1 — Kafka Fundamentals
3. Chapter 2 — Topics, Partitions & Offsets
4. Chapter 2.5 — Internal Mechanics Deep Dive
5. Chapter 3 — Producers & Message Keys
6. Chapter 3.5 — Producer Idempotence Management
7. Chapter 4 — Consumers & Consumer Groups
8. Chapter 4.5 — End-to-End Idempotent Consumer Solution
9. Chapter 5 — Brokers, Clusters & KRaft
10. Chapter 6 — Consolidated Java Producer & Consumer Code
11. Chapter 7 — Offset Management & Delivery Semantics
12. Chapter 8 — Cloud Setup (Conduktor Playground)
13. Chapter 9 — Kafka Security Management
14. Capstone Project — Wikimedia Real-Time Streaming Pipeline
15. Appendix — Master CLI Cheat Sheet


---

## Master Architecture Overview

This is the “zoomed-out” picture of everything covered in this guide — how a message travels from a Producer, through the Cluster (protected by Security), into Consumers/Streams, and finally into a Sink system, with the Controller/KRaft quorum managing metadata the whole time.




*Master Architecture Overview*

*Figure: Master Architecture Overview*

**What each arrow means:**

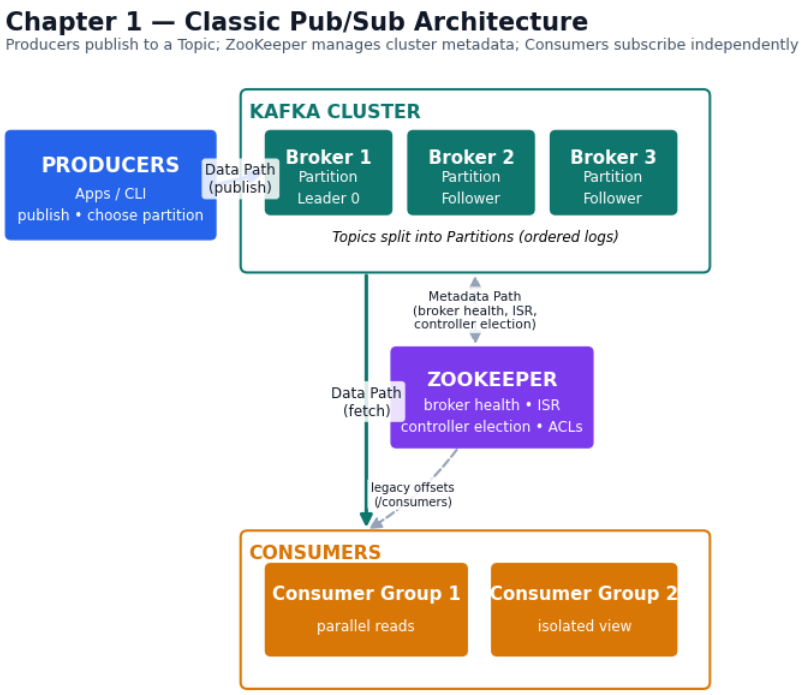
- **(1) Producer -> Cluster:** Records are serialized, routed by the Partitioner (key hash or sticky partitioning), batched, and sent to the partition **Leader** broker.
- **Broker <-> Broker (ISR replication):** Followers continuously fetch from the Leader to stay "in sync" (ISR). The Controller promotes a Follower to Leader automatically on failure.
- **Controller / KRaft:** The elected metadata authority. It watches broker liveness and commits leadership changes to the internal `__cluster_metadata` topic (or ZooKeeper in the classic model).
- **(2) Cluster -> Consumers:** Consumers in a group divide partitions among themselves 1:1 and pull (poll) records; committed read-positions are persisted in `__consumer_offsets`.
- **(3) Consumers -> Sinks:** Processed data is written idempotently downstream (DB upsert, search index, etc.) to guarantee no double-effects even if Kafka delivers duplicates.
- **Security wrapper:** Every hop above is wrapped in TLS encryption, SASL/mTLS authentication, and ACL authorization before any read/write is permitted.

## Chapter 1 — Kafka Fundamentals

### 1.1 Definition

Apache Kafka is a **distributed event streaming platform** designed for high-throughput, fault-tolerant, publish-subscribe messaging. Unlike traditional message queues (which delete messages after consumption), Kafka is a **durable log**. It retains messages based on a time/size retention policy, allowing consumers to "replay" past events. It decouples data pipelines, enabling asynchronous communication between services.

### 1.2 Architecture Diagram — Classic Setup (Producers → Cluster → Consumers, ZooKeeper Metadata)




*Chapter 1 - Classic Pub/Sub Architecture*

*Figure: Chapter 1 - Classic Pub/Sub Architecture*

**Flow meaning:**

1. **Producers -> Brokers (Data Path):** Producers publish records to a Topic; the message lands on whichever broker is the **Leader** for the target partition.
2. **Brokers <-> ZooKeeper (Metadata Path):** Brokers register themselves and continuously sync broker health, In-Sync-Replica (ISR) lists, and controller election state with ZooKeeper.

3. **Brokers -> Consumers (Data Path):** Consumers fetch records from the partition Leader and, in the classic model, historically stored consumer offsets under ZooKeeper's `/consumers` path (modern Kafka stores these in the `__consumer_offsets` topic instead — see Chapter 4).

### 1.3 Publish-Subscribe Architecture Overview

In Kafka's implementation of pub/sub:

- **Producers** publish messages to **Topics**. They don't care who reads them.
- **Topics** are divided into **Partitions** (ordered, immutable sequences of records). Each record gets a unique **Offset**.
- **Consumers** subscribe to Topics and work within **Consumer Groups**. If you have 4 partitions and 4 consumers in the same group, each consumer reads exactly 1 partition (parallelism). If you have 2 groups, both groups read all 4 partitions independently (broadcast).
- **ZooKeeper** (in the classic setup) holds the crucial "source of truth": which broker is the controller, which replica is the ISR leader, and stores `/brokers/ids` and `/consumers` (offsets in older versions).

### 1.4 Real-Time Usage (Industry Use Cases)

- **Event-Driven Microservices:** Service A publishes an "Order Placed" event. Payment, Inventory, and Notification services consume it independently.
- **Real-Time Analytics Pipelines:** Ingesting clickstream data from websites into Kafka, then feeding it into Apache Flink/Spark for real-time dashboards.
- **Log Aggregation:** Centralizing logs from thousands of servers into a single Kafka topic, which then sinks into Elasticsearch for observability.
- **Change Data Capture (CDC):** Using Debezium to capture row-level changes from a PostgreSQL database and publishing them to Kafka, keeping caches/search indexes in sync instantly.

### 1.5 Hands-On: Your First Kafka Interaction (CLI)

**Step 1 — Start ZooKeeper (Metadata service):**

```
bin/zookeeper-server-start.sh config/zookeeper.properties
```

Wait for it to bind to port 2181.

**Step 2 — Start Kafka Broker (Data service):**

```
bin/kafka-server-start.sh config/server.properties
```

Wait for it to register the broker ID with ZooKeeper.

**Step 3 — Create a Topic (Logical channel):**

```
bin/kafka-topics.sh --create --topic quickstart-events \
  --bootstrap-server localhost:9092 \
  --partitions 3 --replication-factor 1
```

**Step 4 — Start a Producer (Write data):**

```
bin/kafka-console-producer.sh --topic quickstart-events \
  --bootstrap-server localhost:9092
```

Type a few messages:

```
> Hello Kafka!
> This is my first event.
> Zookeeper is managing the leader for partition 0 right now.
```

**Step 5 — Start a Consumer (Read data):**

```
bin/kafka-console-consumer.sh --topic quickstart-events \
  --bootstrap-server localhost:9092 \
  --group group-1 \
  --from-beginning
```

> **Pro-Tip:** Open another consumer in a third terminal using `--group group-2`. Both consumers read all messages. Open a fourth consumer using `--group group-1`. Now the 3 partitions are split between the two consumers in `group-1` — achieving parallel load balancing.

### 1.6 A Note on the Future: KRaft (No ZooKeeper)

As of Apache Kafka 3.0+, ZooKeeper is officially deprecated for metadata management and will be removed entirely in Kafka 4.0. In modern Kafka setups (**KRaft — Kafka Raft**), ZooKeeper is replaced by a built-in Raft quorum within the Kafka brokers themselves. They now store metadata directly in internal topics (like `__cluster_metadata`), simplifying the architecture by removing an external dependency. However, the core Publish-Subscribe data flow (Producers → Brokers → Consumers) remains completely unchanged — only the metadata plane has evolved. (See Chapter 5.)

---

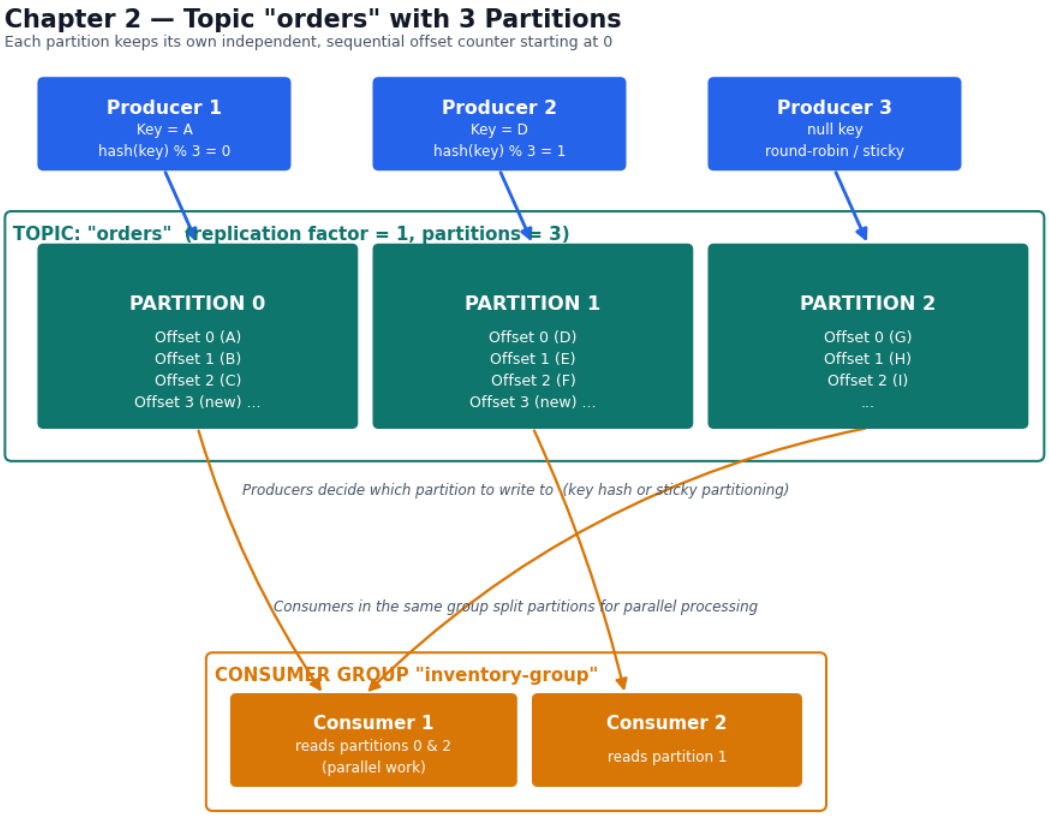
## Chapter 2 — Topics, Partitions & Offsets

### 2.1 Definition

- **Topic:** A logical category or feed name to which records are published. Similar to a table in a database, but without schema constraints.
- **Partition:** A physical, ordered, immutable sequence of records. A Topic is split into Partitions to allow parallel processing.
- **Offset:** A unique, sequential integer ID assigned to each record within a specific partition. Offsets are immutable — once a record is written, its offset never changes.

**Crucial Rule:** Kafka guarantees order strictly *within* a single partition, but not *across* partitions in the same topic. If you need total order for all events, you must use only 1 partition (sacrificing parallelism).

## 2.2 Architectural Flow Diagram — 3-Partition Topic Visual




*Chapter 2 - Topic “orders” with 3 Partitions*

*Figure: Chapter 2 - Topic “orders” with 3 Partitions*

**Key points from the visual:**

- Offsets restart at 0 for **each** partition — they are not global across the topic.
- Records are strictly ordered within Partition 0 (Offset 0 → 1 → 2 → 3).
- If Producer 1 sends 100 records with the same Key, all 100 go to the same partition, guaranteeing their order.

## 2.3 Hands-On: Essential CLI Commands for Topics

**A. Create a Topic:**

```
bin/kafka-topics.sh --create \
  --topic orders \
  --bootstrap-server localhost:9092 \
```

```
  --partitions 3 \
  --replication-factor 1
```

> Use `--replication-factor 1` only for single-broker dev. In production, use 3.

**B. List All Topics:**

```
bin/kafka-topics.sh --list --bootstrap-server localhost:9092
```

**C. Describe a Topic (partitions, leaders, ISR, configs):**

```
bin/kafka-topics.sh --describe --topic orders --bootstrap-server
localhost:9092
```

Sample output:

```
Topic: orders  TopicId: abc123  PartitionCount: 3  ReplicationFactor:
1  Configs: segment.bytes=1073741824
  Partition: 0  Leader: 1  Replicas: 1  Isr: 1
  Partition: 1  Leader: 1  Replicas: 1  Isr: 1
  Partition: 2  Leader: 1  Replicas: 1  Isr: 1
```

(`Isr` = In-Sync Replicas; if a broker fails, the leader changes here.)

**D. Alter Partitions (increase only — Kafka does NOT allow decreasing):**

```
bin/kafka-topics.sh --alter --topic orders \
  --bootstrap-server localhost:9092 \
  --partitions 6
```

> Warning: Partition rebalancing does not redistribute existing data automatically — only new data follows the new partition count.

**E. View Offsets Directly:**

```
bin/kafka-run-class.sh kafka.tools.GetOffsetShell \
  --topic orders --bootstrap-server localhost:9092
```

Output shows the latest offset for each partition (e.g. `orders:0:5` means Partition 0 has written up to offset 4, next is 5).

**F. Produce Messages with Keys (control partition routing):**

```
bin/kafka-console-producer.sh --topic orders \
  --bootstrap-server localhost:9092 \
  --property parse.key=true \
  --property key.separator=:
```

Type: `user_123:{"item":"laptop"}` → the hash of `user_123` routes it to a fixed partition, ensuring all orders from `user_123` land in the same partition (maintaining order).

**G. Consume from a Specific Partition & Offset:**

```
bin/kafka-console-consumer.sh --topic orders \
  --bootstrap-server localhost:9092 \
  --partition 0 \
  --offset 2 \
  --group manual-reader
```

This reads only Partition 0, starting at Offset 2 (skipping the first two events).

### 2.4 Real-Time Usage

- **Ordering Guarantee (Finance):** Process events for Account A in exact chronological order by using `Account_ID` as the message key.
- **Horizontal Scaling:** A social platform splits `user-actions` into 50 partitions and runs 50 consumer instances in one group — 50x the throughput.
- **Immutable Offsets for Replay (Data Warehousing):** If an ETL pipeline crashes, reset the consumer group's committed offset back and replay the exact same immutable records.
- **Storage Tiering:** Retention (e.g. 7 days) auto-deletes old partition segments, giving a bounded, manageable storage footprint.

### 2.5 Critical Internal Mechanics (Controller & Partition Leaders)

Every partition has exactly one **Leader** (the broker handling all reads/writes for that partition). The **Controller** (elected by ZooKeeper/KRaft) monitors partition health. If a broker crashes, the Controller immediately notices, elects a new Leader from the ISR list, and updates the metadata — achieving zero data loss and high availability with zero manual intervention.

---

## Chapter 2.5 — Internal Mechanics Deep Dive

### How Topics, Partitions & Offsets Actually Work On Disk

#### *1. Physical Storage Internals: A Partition Is NOT a Single File*

When you create a topic with 3 partitions, Kafka does **not** create one giant file per partition. Instead, each partition is split into **Segments** on disk.

**Directory Structure (Linux):** `/var/lib/kafka/data/topic-orders/partition-0/`

```
00000000000000000000.log        (the actual message data)
00000000000000000000.index      (offset -> byte position mapping)
00000000000000000000.timeindex  (timestamp -> offset mapping)
00000000000000000123.log        (segment 2, starting at offset 123)
00000000000000000123.index
...
```

**Why segments?** If you had a single 1 TB file, deleting old messages would be impossibly slow. Kafka rolls over to a new segment when the current one hits `log.segment.bytes` (default

1GB) or `log.roll.hours` (default 7 days). When retention expires, Kafka deletes entire segment files in O(1) time — just an unlink operation.

### *2. The Magic of the Index File (Offset Lookup)*

The `.log` file contains raw messages of variable size. The `.index` file is a **sparse** map (not a full index, to save memory) that maps every N bytes (`log.index.interval.bytes`, default 4096) to the exact byte position in the `.log` file.

**Lookup flow (binary search + scan):**

1. Kafka binary-searches the `.index` file for the closest entry before the target offset.
2. It jumps directly to that byte position in the `.log` file.
3. It scans sequentially the small remaining distance until it finds the exact message.

```
LOOKUP DIAGRAM: "Find Offset 5000 in Partition 0"

  .index file (sparse)                    .log file (raw messages)
  +------------------+
+-------------------------------+

  | Offset 4990       | ---byte 1,024,000-->| ... [msg 4990][msg
4991]...    |
  +------------------+                    |     [msg 4999][msg 5000]
<-- found after

                                             |      a short sequential
scan   |

+-------------------------------+
```

Because of the sparse index, disk seeks are minimized and the final scan is tiny.

### *3. Replication Internals: LEO and HW*

- **LEO (Log End Offset):** The offset of the next message that will be written to a replica.
- **HW (High Watermark):** The highest offset committed to ALL ISR replicas. Consumers can only read up to the HW — anything beyond is "uncommitted" and invisible.

**Write path (with `acks=all`):**

1. Producer sends a batch to the Partition Leader.
2. Leader appends to its local `.log` and updates its LEO immediately.
3. Leader forwards to all Follower brokers in the ISR.
4. Followers write to their own logs and update their LEO, then ACK back to the Leader.
5. When all ISR followers report the new LEO, the Leader advances the HW.
6. Only then does the Leader ACK the Producer.

If the Leader crashes, the Controller picks the Follower with the highest LEO still within the ISR — since it had caught up to the HW, **zero data is lost** (given correct `min.insync.replicas`).

### *4. Zero-Copy & Page Cache (Why Kafka Is So Fast)*

- **OS Page Cache:** Writes land in the Linux page cache (RAM) first — most writes are just memory copies, no `fsync` per message.
- **Zero-Copy Reads:** Consumer fetches use the `sendfile()` syscall, copying directly from Page Cache → Socket Buffer (NIC), bypassing the JVM heap entirely. This lets Kafka saturate 10 GbE networks on standard hardware.

### *5. Log Compaction (Key-Based Cleanup)*

Unlike time/size-based retention, **Log Compaction** deletes based on keys. A background "Cleaner" thread rewrites `.log` segments, keeping only the latest message per key — vital for `KTable` state restoration in Kafka Streams.

### *6. Consumer Offset Management: `__consumer_offsets`*

Committed offsets are stored in an internal, compacted topic `__consumer_offsets` (50 partitions by default). Key = `GroupID + Topic + Partition`; Value = committed offset. Because it's log-compacted, only the latest committed offset per group/partition survives.

### *7. Partition Assignment Algorithms*

- **Range Assignor (pre-3.0 default):** Contiguous ranges — prone to skew.
- **Round-Robin Assignor:** Circular one-by-one assignment.
- **Sticky Assignor (modern default):** Even distribution while minimizing partition movement on rebalance.

### *8. Controller Election Internals*

In the classic ZooKeeper setup, the Controller watches `/brokers/ids`. On a crash it: (1) reads the ISR list for affected partitions, (2) updates Leader metadata to a new ISR member, (3) sends `LeaderAndIsr` RPCs to all alive brokers. Failover typically completes in under 5 seconds. In KRaft mode this logic is internalized via Raft consensus — the mechanics are identical, minus ZooKeeper.

**Summary Table:**

| Component | Internal Reality |
|---|---|
| Topic | A logical namespace; physically just a folder containing partition directories |
| Partition | A directory of immutable segment files (`.log` + `.index` + `.timeindex`) |
| Offset | An 8-byte signed long; maps to a physical byte position via sparse indexing |
| Write | Append to leader's page cache; wait for ISR LEOs; advance HW; ACK |
| Read | Binary-search the index, `sendfile()` zero-copy stream to the network |

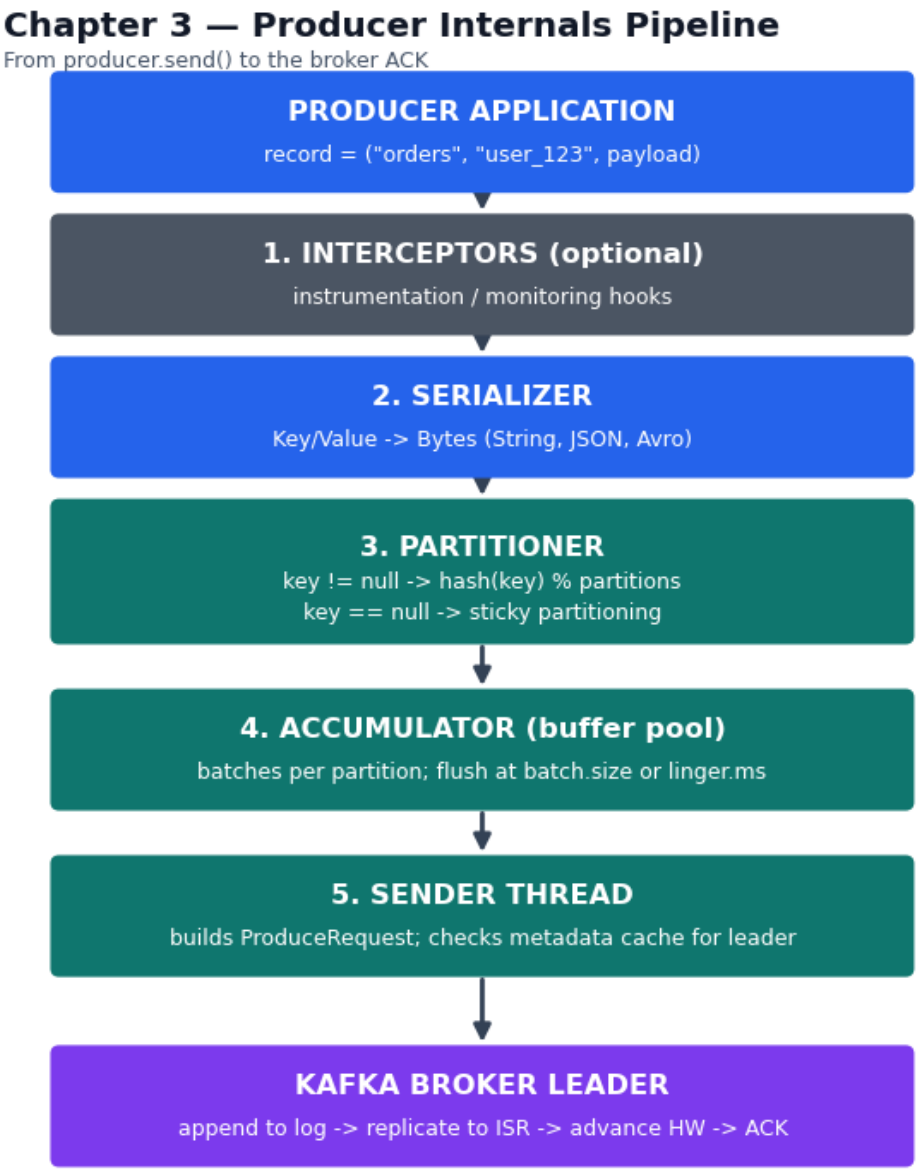
| Component | Internal Reality |
| --- | --- |
| Retention | Delete old segment files (time/size) OR compact the log (key-based) |
| Offset Commit | Stored as a message in `__consumer_offsets` (compacted), not in-memory |

## Chapter 3 — Producers & Message Keys

### 3.1 Definition

- **Producer:** A client application that publishes records to one or more Kafka Topics.
- **Message Key:** An optional byte array in a record. The Producer uses the key to determine partition placement via hashing. If a key is `null`, Kafka distributes records via sticky partitioning.
- **Partitioner:** The logic deciding which partition number a record goes to.
- **Batch:** A collection of records destined for the same topic-partition, buffered together to amortize network round-trip time.

### 3.2 Architectural Flow Diagram — The Producer Pipeline




*Chapter 3 - Producer Internals Pipeline*

*Figure: Chapter 3 - Producer Internals Pipeline*

### 3.3 Deep Dive: Sticky Partitioning

**Old way (pre-2.4):** Round-robin per record when `key=null` → many small, slow-filling batches.

**Modern way (Sticky Partitioner, default):** When `key=null`, the Producer picks one partition and sticks to it until that batch fills (`batch.size`) or `linger.ms` expires, then switches. This fills 16KB batches faster, reduces `ProduceRequest` count, and can improve throughput by up to ~30% versus round-robin.

### 3.4 The `acks` Tradeoffs (Durability vs. Latency)

| Setting | Description | Waits For | Data Loss Risk | Throughput |
|---|---|---|---|---|
| `acks=0` | Fire-and-forget | Nothing | Extremely High | Maximum |
| `acks=1` | Leader ACK only | Leader's local write | Moderate | High |
| `acks=all` (-1) | ISR ACK | Leader + all ISR | Zero (with `min.insync.replicas`) | Lowest |

> **Critical combo:** `acks=all` + `enable.idempotence=true` + `max.in.flight.requests.per.connection<=5` gives Exactly-Once semantics per partition by eliminating duplicate retries and ensuring no data loss.

### 3.5 Hands-On: Producer Configurations & CLI

**A. Java Producer Setup (with keys and `acks=all`):**

```
Properties props = new Properties();
props.put("bootstrap.servers", "localhost:9092");
props.put("key.serializer",
"org.apache.kafka.common.serialization.StringSerializer");
props.put("value.serializer",
"org.apache.kafka.common.serialization.StringSerializer");

props.put("acks", "all");
props.put("enable.idempotence", "true");
props.put("max.in.flight.requests.per.connection", "5");

Producer<String, String> producer = new KafkaProducer<>(props);

producer.send(new ProducerRecord<>("orders", "user_123",
"{'item':'laptop'}"));
producer.send(new ProducerRecord<>("orders", null,
"{'item':'mouse'}")); // sticky partition
```

**B. CLI: Producing with Keys**

```
bin/kafka-console-producer.sh --topic orders \
  --bootstrap-server localhost:9092 \
  --property parse.key=true \
  --property key.separator=:

user_456:{"event":"login"}
user_456:{"event":"click"}   # same partition -> order guaranteed
null:{"event":"heartbeat"}   # sticky partition
```

**C. CLI: Custom Partitioner**

```
props.put("partitioner.class", "com.myapp.CustomPartitioner");
```

### 3.6 Real-Time Usage

- **Multi-Tenant SaaS Ordering:** Key = `tenant_id + user_id`; `acks=all` with `min.insync=2` guarantees an "Order Placed" event is never lost before invoicing.
- **IoT Sensor Ingestion:** Key = `null`, `acks=0` — losing a single reading doesn't matter; sub-5ms wire-speed writes for millions of sensors.
- **Database CDC:** Key = source primary key, `acks=all` — losing a DR replica is unacceptable.

### 3.7 Critical Producer Tuning Parameters

| Config | Default | Meaning |
|---|---|---|
| `buffer.memory` | 32MB | Total memory available to batch records |
| `batch.size` | 16KB | Max bytes per single partition batch |
| `linger.ms` | 0 | Time to wait before flushing an unfilled batch |
| `max.in.flight.requests.per.connection` | 5 | Unacknowledged batches allowed per connection |

## Chapter 3.5 — Producer Idempotence Management

### End-to-End Hands-On Code + Idempotence Internals

**A. Maven Dependency:**

```
<dependency>
    <groupId>org.apache.kafka</groupId>
    <artifactId>kafka-clients</artifactId>
    <version>3.6.0</version>
</dependency>
```

**B. Production-Ready Producer (`IdempotentOrderProducer.java`):**

```java
package com.kafka.producer;

import org.apache.kafka.clients.producer.*;
import org.apache.kafka.common.serialization.StringSerializer;
import java.util.Properties;

public class IdempotentOrderProducer {
    private static final String TOPIC = "orders";

    public static void main(String[] args) {
        Properties props = new Properties();
        props.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG,
"localhost:9092");
        props.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG,
StringSerializer.class.getName());
        props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG,
StringSerializer.class.getName());

        // CRITICAL: idempotence + acks + in-flight requests
        props.put(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, "true");
        props.put(ProducerConfig.ACKS_CONFIG, "all");

props.put(ProducerConfig.MAX_IN_FLIGHT_REQUESTS_PER_CONNECTION, "5");
        props.put(ProducerConfig.RETRIES_CONFIG, Integer.MAX_VALUE);
        props.put(ProducerConfig.DELIVERY_TIMEOUT_MS_CONFIG, 120000);
        props.put(ProducerConfig.REQUEST_TIMEOUT_MS_CONFIG, 30000);
        props.put(ProducerConfig.COMPRESSION_TYPE_CONFIG, "snappy");

        try (KafkaProducer<String, String> producer = new
KafkaProducer<>(props)) {
            for (int i = 0; i < 10; i++) {
                String key = "user_123";
                String value = "{\"orderId\": " + i + ", \"amount\": "
+ (i * 100) + "}";
                ProducerRecord<String, String> record = new
ProducerRecord<>(TOPIC, key, value);

                producer.send(record, (metadata, exception) -> {
                    if (exception != null) {
                        System.err.println("Send failed for order " +
i + ": " + exception.getMessage());
                    } else {
                        System.out.printf("Order %d sent. Partition:
%d, Offset: %d%n",
                                i, metadata.partition(),
metadata.offset());
                    }
                });
            }
            producer.flush();
```

```
        }
    }
}
```

### How Idempotence Works Internally

| Component | Mechanism |
|---|---|
| Producer ID (PID) | Assigned by broker on startup; valid for the session |
| Sequence Numbers | Monotonically increasing per (PID, partition) |
| Broker-side cache | `(PID, Partition) -> LastAcknowledgedSequenceNumber` |
| Validation | `seq == last+1` -> accept; `seq == last` -> dedupe/ACK; `seq < last` -> reject; gap -> reject |
| Retry behavior | Retries reuse the same sequence number, so the broker recognizes duplicates safely |

### Configuration Constraints Enforced by `enable.idempotence=true`

| Setting | Forced Value | Why |
|---|---|---|
| `acks` | `all` | Must confirm all replicas before trusting the dedupe cache |
| `retries` | `Integer.MAX_VALUE` | Must retry indefinitely to avoid sequence gaps |
| `max.in.flight.requests` | `<= 5` | Preserves ordering across parallel in-flight batches |

### The "Idempotence Pitfall"

Idempotence only protects against **Producer-induced** duplicates from retries. It does **not** protect against:

- **Manual duplicates:** calling `send()` twice for the same event in application code.
- **Cross-session duplicates:** a crashed Producer restarting with a new PID while old, pending messages eventually land. (Solution: Transactional Producers + Idempotence.)

### Production Checklist (broker `server.properties`)

```
unclean.leader.election.enable=false
min.insync.replicas=2
```

---

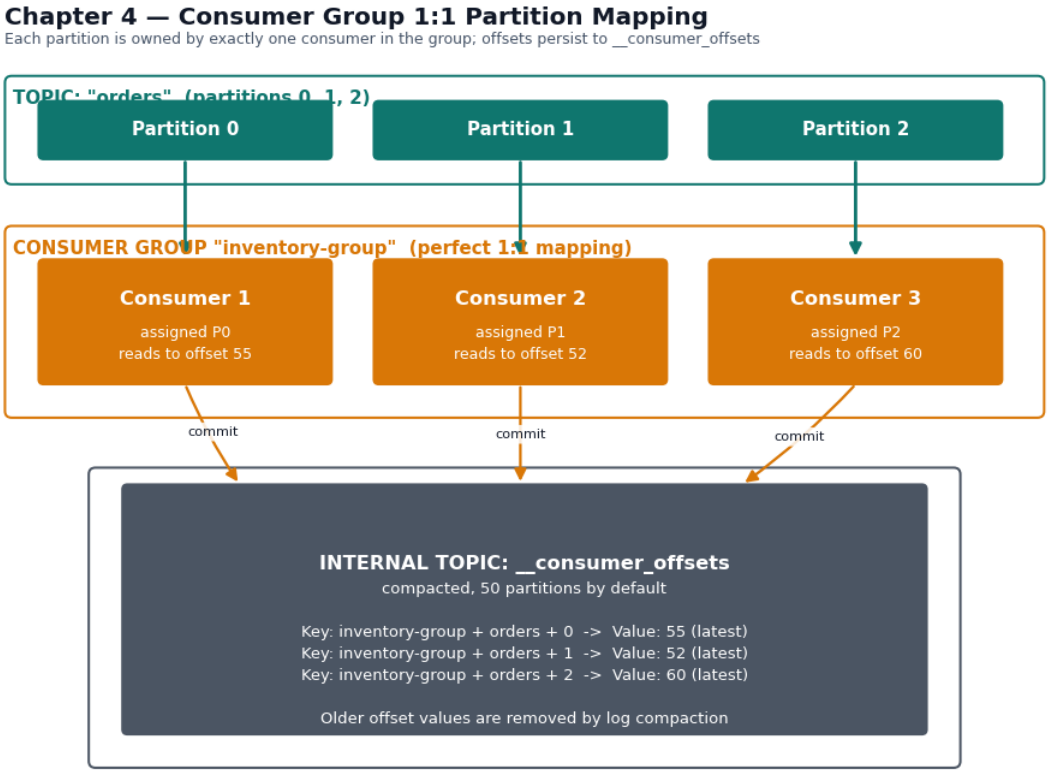
## Chapter 4 — Consumers & Consumer Groups

### 4.1 Definition

- **Consumer:** A client application that reads records from Kafka topics.

- **Consumer Group:** A set of consumers working together; Kafka guarantees each partition maps to exactly one consumer in the group (1:1) for parallel processing.
- **Group Coordinator:** A broker managing group membership, rebalances, and offset commits.
- **`__consumer_offsets`:** An internal, compacted topic storing committed offsets per (Group, Topic, Partition).
- **Partition Assignment:** Distribution of partitions among consumers; excess consumers idle.

### 4.2 Architectural Flow Diagram — 1:1 Mapping & Offset Storage




*Chapter 4 - Consumer Group 1:1 Partition Mapping*

*Figure: Chapter 4 - Consumer Group 1:1 Partition Mapping*

If a 4th consumer joins, a **rebalance** occurs and 3 partitions split across 4 consumers — one sits idle. If a consumer crashes, its partitions reassign to others within seconds.

### 4.3 Handling Failed Messages & Poison Pills

A **poison pill** is a message the consumer can't process (bad JSON, logic error). Letting the exception propagate stops the poll loop and eventually triggers a rebalance. Production solutions:

1. **Catch, log, and commit (skip)** — for non-critical data.
2. **Dead Letter Queue (DLQ)** — send raw failed message to a separate topic.
3. **Pause & Resume** — stop the flood if the issue is systemic (e.g. downstream DB down).

```
try {
    processOrder(record.value());
} catch (JsonProcessingException e) {
    System.err.printf("Poison Pill! Topic: %s, Partition: %d, Offset:
%d, Value: %s%n",
            record.topic(), record.partition(), record.offset(),
record.value());
    consumer.commitSync(Map.of(
        new TopicPartition(record.topic(), record.partition()),
        new OffsetAndMetadata(record.offset() + 1)
    ));
    // Option B: deadLetterProducer.send(new ProducerRecord<>("orders-
dlq", record.value()));
}
```

### 4.4 Consumer-Side Idempotency (Exactly-Once Processing)

Producer idempotence prevents duplicates from **retries**; it does **not** prevent a consumer from seeing duplicates on rebalance (offset commit failure, then re-read). Solutions:

- **Idempotent writes:** DB upserts keyed on `event_id`.
- **External dedup cache:** check Redis for `event_id` before processing.
- **`isolation.level=read_committed`:** masks aborted transactions when paired with transactional/idempotent producers.

### 4.5 Partition Issues: Rebalancing & Stuck Consumers

**Problem — processing takes too long:** the consumer must call `poll()` within `max.poll.interval.ms` (default 5 min) or the Coordinator assumes it's dead and rebalances. **Fix:** increase the interval, or use async background processing while still polling frequently.

**Problem — rebalance storms on restart:** use `group.instance.id` for static membership so consumers can rejoin without immediately dropping their partitions.

```
bin/kafka-consumer-groups.sh --describe --group inventory-group --
bootstrap-server localhost:9092

GROUP           TOPIC  PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG
CONSUMER-ID     HOST
inventory-group orders 0          55              60              5
```

```
consumer-1      /192.168.1.10
inventory-group orders 1          52              52              0
consumer-2      /192.168.1.11
inventory-group orders 2          60              63              3
consumer-3      /192.168.1.12
```

### 4.6 Hands-On: Essential Consumer CLI Commands

```
# Basic consume
bin/kafka-console-consumer.sh --topic orders --bootstrap-server
localhost:9092 --group inventory-group

# From specific partition & offset
bin/kafka-console-consumer.sh --topic orders --bootstrap-server
localhost:9092 --partition 0 --offset 10

# Reset offsets - dry run
bin/kafka-consumer-groups.sh --group inventory-group --bootstrap-
server localhost:9092 --to-earliest --dry-run

# Reset offsets - execute
bin/kafka-consumer-groups.sh --group inventory-group --bootstrap-
server localhost:9092 --to-earliest --execute

# Reset to timestamp
bin/kafka-consumer-groups.sh --group inventory-group --bootstrap-
server localhost:9092 \
  --to-datetime 2026-07-12T12:00:00.000 --execute

# Reset a specific partition to an offset
bin/kafka-consumer-groups.sh --group inventory-group --bootstrap-
server localhost:9092 \
  --to-offset 50 --topic orders:0 --execute

# List / delete groups
bin/kafka-consumer-groups.sh --list --bootstrap-server localhost:9092
bin/kafka-consumer-groups.sh --group inventory-group --bootstrap-
server localhost:9092 --delete
```

### 4.7 Real-Time Usage

- **Fraud Detection:** `isolation.level=read_committed` + Redis `SETNX transaction_id` for idempotent processing.
- **Log Shipping to Elasticsearch:** poison pills are caught, logged, and shipped to a `dead-logs` topic instead of crashing the pipeline.
- **IoT Dashboard Rebuild:** adding partitions triggers automatic rebalance and reassignment.

### 4.8 Crucial Consumer Configuration Summary

| Config | Default | Recommendation |
|---|---|---|
| `group.id` | null | Mandatory |
| `enable.auto.commit` | true | Set `false` in production |
| `auto.offset.reset` | latest | `earliest` is safer for recovery |
| `max.poll.records` | 500 | Lower for heavy processing |
| `isolation.level` | read_uncommitted | `read_committed` for EOS pipelines |
| `partition.assignment.strategy` | RangeAssignor | `CooperativeStickyAssignor` for minimal rebalance pauses |

## Chapter 4.5 — End-to-End Idempotent Consumer Solution

**`Order.java` (shared POJO)**

```
package com.kafka.model;

public class Order {
    private String orderId;
    private int amount;
    public Order() {}
    public Order(String orderId, int amount) { this.orderId = orderId;
this.amount = amount; }
    public String getOrderId() { return orderId; }
    public void setOrderId(String orderId) { this.orderId = orderId; }
    public int getAmount() { return amount; }
    public void setAmount(int amount) { this.amount = amount; }
    @Override public String toString() { return "Order{id='" + orderId
+ "', amount=" + amount + "}"; }
}
```

**`IdempotentOrderConsumer.java`**

```
package com.kafka.consumer;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.kafka.model.Order;
import org.apache.kafka.clients.consumer.*;
import org.apache.kafka.common.TopicPartition;
import org.apache.kafka.common.serialization.StringDeserializer;
import java.time.Duration;
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;

public class IdempotentOrderConsumer {
    private static final String TOPIC = "orders";
```

```
    private static final String GROUP_ID = "inventory-group";
    private static final ConcurrentHashMap<String, Boolean>
PROCESSED_STORE = new ConcurrentHashMap<>();
    private static final Map<TopicPartition, Long> CURRENT_OFFSETS =
new ConcurrentHashMap<>();
    private static final ObjectMapper objectMapper = new
ObjectMapper();

    public static void main(String[] args) {
        Properties props = new Properties();
        props.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG,
"localhost:9092");
        props.put(ConsumerConfig.GROUP_ID_CONFIG, GROUP_ID);
        props.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG,
StringDeserializer.class.getName());
        props.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG,
StringDeserializer.class.getName());
        props.put(ConsumerConfig.ENABLE_AUTO_COMMIT_CONFIG, "false");
        props.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG,
"earliest");
        props.put(ConsumerConfig.MAX_POLL_RECORDS_CONFIG, "10");
        props.put(ConsumerConfig.ISOLATION_LEVEL_CONFIG,
"read_committed");
        props.put(ConsumerConfig.PARTITION_ASSIGNMENT_STRATEGY_CONFIG,

"org.apache.kafka.clients.consumer.CooperativeStickyAssignor");

        try (KafkaConsumer<String, String> consumer = new
KafkaConsumer<>(props)) {
            ConsumerRebalanceListener rebalanceListener = new
ConsumerRebalanceListener() {
                @Override public void
onPartitionsAssigned(Collection<TopicPartition> partitions) {
                    System.out.println("Partitions Assigned: " +
partitions);
                }
                @Override public void
onPartitionsRevoked(Collection<TopicPartition> partitions) {
                    System.out.println("Partitions Revoked: " +
partitions + ". Committing final offsets...");
                    consumer.commitSync(CURRENT_OFFSETS);
                }
            };
            consumer.subscribe(Collections.singletonList(TOPIC),
rebalanceListener);
            System.out.println("Consumer started. Listening to topic:
" + TOPIC);

            while (true) {
                ConsumerRecords<String, String> records =
```

```
consumer.poll(Duration.ofMillis(1000));
                if (records.isEmpty()) continue;

                Map<TopicPartition, Long> offsetsToCommit = new
HashMap<>();
                for (ConsumerRecord<String, String> record : records)
{
                    TopicPartition tp = new
TopicPartition(record.topic(), record.partition());
                    try {
                        Order order =
objectMapper.readValue(record.value(), Order.class);
                        if
(PROCESSED_STORE.containsKey(order.getOrderId())) {
                            System.out.printf("DUPLICATE SKIPPED: P%d
O%d id=%s%n",
                                    record.partition(),
record.offset(), order.getOrderId());
                        } else {
                            System.out.printf("PROCESSED: P%d O%d
order=%s%n",
                                    record.partition(),
record.offset(), order);
                            PROCESSED_STORE.put(order.getOrderId(),
true);
                        }
                        offsetsToCommit.put(tp, record.offset());
                    } catch (Exception e) {
                        System.err.printf("POISON PILL! P%d O%d value=
%s%n",
                                record.partition(), record.offset(),
record.value());
                        offsetsToCommit.put(tp, record.offset());
                    }
                }

                if (!offsetsToCommit.isEmpty()) {
                    Map<TopicPartition, OffsetAndMetadata> commitMap =
new HashMap<>();
                    for (Map.Entry<TopicPartition, Long> entry :
offsetsToCommit.entrySet()) {
                        long nextOffset = entry.getValue() + 1;
                        commitMap.put(entry.getKey(), new
OffsetAndMetadata(nextOffset));
                        CURRENT_OFFSETS.put(entry.getKey(),
nextOffset);
                    }
                    consumer.commitSync(commitMap);
                    System.out.printf("Committed offsets: %s%n",
commitMap);
```

```
                }
            }
        }
    }
}
```

### Test Scenarios

1. **Create topic:** `bin/kafka-topics.sh --create --topic orders --bootstrap-server localhost:9092 --partitions 3 --replication-factor 1`
2. **Run the consumer**, then produce a mix of good and bad JSON:

```
{"orderId":"ord-1","amount":100}
{"orderId":"ord-2","amount":200}
{"orderId":"ord-1","amount":150}     <- duplicate orderId (tests
idempotency)
{"badJson": no quotes here           <- POISON PILL (invalid JSON)
{"orderId":"ord-3","amount":300}
```

3. **Observed output:** `ord-1` is processed only once; the poison pill is caught, logged, and skipped (committed offset advances past it); the consumer keeps running.

### Rebalance Test

Start a second consumer in the same group; watch Consumer 1 log `Partitions Revoked` (with a final commit) then `Partitions Assigned` with fewer partitions — no data is lost because the listener commits before losing ownership.

### Monitoring & Recovery CLI

```
bin/kafka-consumer-groups.sh --describe --group inventory-group --
bootstrap-server localhost:9092
bin/kafka-consumer-groups.sh --group inventory-group --bootstrap-
server localhost:9092 --to-offset 0 --topic orders:0 --execute
bin/kafka-consumer-groups.sh --group inventory-group --bootstrap-
server localhost:9092 --delete
```

### Production Checklist

- No data loss (manual `commitSync()` after successful processing).
- Fault tolerance (poison pills logged and skipped, not fatal).
- Exactly-once processing (idempotent store + `read_committed`).
- Graceful shutdowns (rebalance listener commits before revocation).
- Observability (`--describe` for lag monitoring).

---

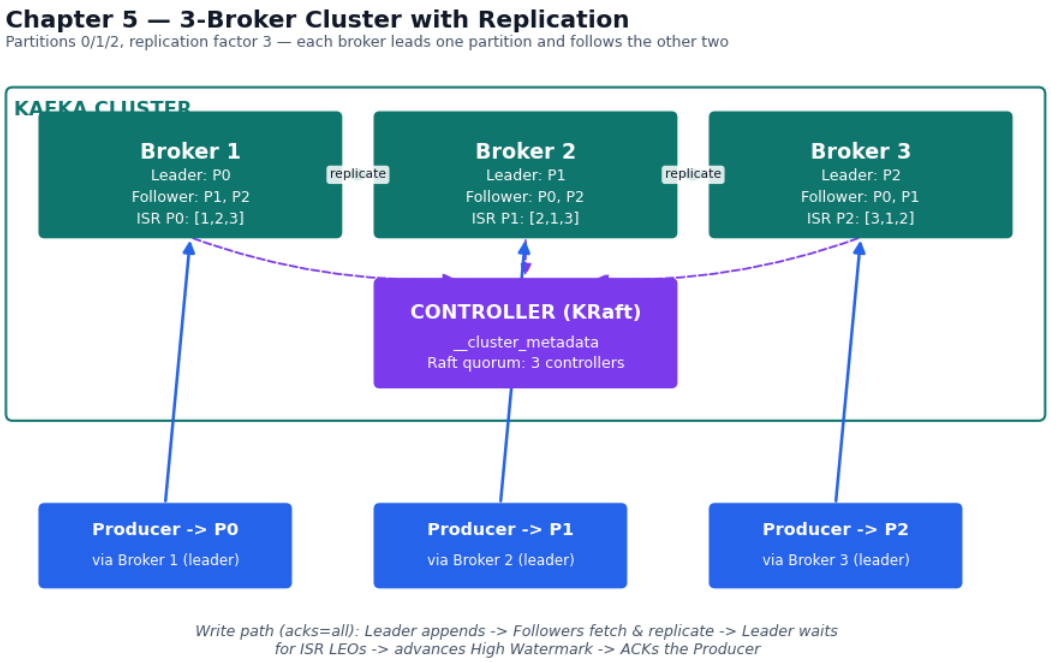
## Chapter 5 — Brokers, Clusters & KRaft

### 5.1 Definition

- **Broker:** A single Kafka server process (JVM). Stores partitions, handles client requests, manages replication.

- **Cluster:** A group of brokers providing fault-tolerance and horizontal scaling.
- **Partition Leader:** The broker handling all reads/writes for a given partition.
- **Partition Follower:** Brokers passively replicating from the Leader; promoted on failure.
- **ISR (In-Sync Replicas):** Replicas fully caught up with the Leader's Log End Offset.
- **Controller:** The elected broker managing cluster metadata, leader elections, and broker membership.
- **KRaft (Kafka Raft):** The Kafka 3.0+ consensus protocol replacing ZooKeeper; metadata lives in the internal `__cluster_metadata` partition, managed by a Raft quorum of controllers.

## 5.2 Architectural Flow Diagram — 3-Broker Cluster with Replication




*Chapter 5 - 3-Broker Cluster with Replication*

*Figure: Chapter 5 - 3-Broker Cluster with Replication*

**The write path across the cluster (`acks=all`):**

1. Producer connects to Broker 1 (Leader for P0).
2. Leader writes the batch to its local page cache, updates its LEO.

3. Followers (Broker 2 & 3) continuously fetch from Broker 1 in the background.
4. Followers write to their own logs, update their LEOs, and report back.
5. Leader waits until all ISR members report the new LEO.
6. Leader advances the High Watermark (HW).
7. Leader ACKs the Producer.

### 5.3 Real-Time Usage

- **High Availability (Finance):** `replication.factor=3` + `min.insync.replicas=2` survives a full broker crash without losing a single transaction.
- **Rack Awareness:** `broker.rack` spreads replicas across AZs; an AZ outage still leaves a healthy ISR to elect from.
- **Rolling Upgrades:** Kill/restart brokers one at a time; the Controller shifts leadership away from the restarting broker for zero downtime.

### 5.4 Hands-On: 3-Node KRaft Cluster (Docker Compose)

```
version: '3'
services:
  kafka1:
    image: confluentinc/cp-kafka:latest
    hostname: kafka1
    container_name: kafka1
    ports: ["9091:9091", "9101:9101"]
    environment:
      KAFKA_NODE_ID: 1
      KAFKA_CONTROLLER_LISTENER_NAMES: 'CONTROLLER'
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:
'CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT,EXTERNAL:PLAINTEXT'
      KAFKA_LISTENERS:
'PLAINTEXT://:9092,CONTROLLER://:9093,EXTERNAL://0.0.0.0:9091'
      KAFKA_ADVERTISED_LISTENERS:
'PLAINTEXT://kafka1:9092,EXTERNAL://localhost:9091'
      KAFKA_CONTROLLER_QUORUM_VOTERS:
'1@kafka1:9093,2@kafka2:9093,3@kafka3:9093'
      KAFKA_PROCESS_ROLES: 'broker,controller'
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 3
      KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 3
      KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 2
      KAFKA_MIN_INSYNC_REPLICAS: 2
      CLUSTER_ID: 'MkU3OEVBNTcwNTJENDM2Qk'
    volumes: ["kafka1-data:/var/lib/kafka/data"]

  kafka2:
    image: confluentinc/cp-kafka:latest
    hostname: kafka2
    container_name: kafka2
    ports: ["9092:9092", "9102:9102"]
```

```
    environment:
      KAFKA_NODE_ID: 2
      KAFKA_CONTROLLER_LISTENER_NAMES: 'CONTROLLER'
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:
'CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT,EXTERNAL:PLAINTEXT'
      KAFKA_LISTENERS:
'PLAINTEXT://:9092,CONTROLLER://:9093,EXTERNAL://0.0.0.0:9092'
      KAFKA_ADVERTISED_LISTENERS:
'PLAINTEXT://kafka2:9092,EXTERNAL://localhost:9092'
      KAFKA_CONTROLLER_QUORUM_VOTERS:
'1@kafka1:9093,2@kafka2:9093,3@kafka3:9093'
      KAFKA_PROCESS_ROLES: 'broker,controller'
      CLUSTER_ID: 'MkU3OEVBNTcwNTJENDM2Qk'
    volumes: ["kafka2-data:/var/lib/kafka/data"]

  kafka3:
    image: confluentinc/cp-kafka:latest
    hostname: kafka3
    container_name: kafka3
    ports: ["9093:9093", "9103:9103"]
    environment:
      KAFKA_NODE_ID: 3
      KAFKA_CONTROLLER_LISTENER_NAMES: 'CONTROLLER'
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:
'CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT,EXTERNAL:PLAINTEXT'
      KAFKA_LISTENERS:
'PLAINTEXT://:9092,CONTROLLER://:9093,EXTERNAL://0.0.0.0:9093'
      KAFKA_ADVERTISED_LISTENERS:
'PLAINTEXT://kafka3:9092,EXTERNAL://localhost:9093'
      KAFKA_CONTROLLER_QUORUM_VOTERS:
'1@kafka1:9093,2@kafka2:9093,3@kafka3:9093'
      KAFKA_PROCESS_ROLES: 'broker,controller'
      CLUSTER_ID: 'MkU3OEVBNTcwNTJENDM2Qk'
    volumes: ["kafka3-data:/var/lib/kafka/data"]

volumes:
  kafka1-data:
  kafka2-data:
  kafka3-data:
```

**Start it:**

```
docker-compose -f docker-compose-kraft.yml up -d
docker ps
```

**Validate the Raft quorum:**

```
docker exec -it kafka1 kafka-metadata-quorum.sh --bootstrap-server
localhost:9092 describe --status
```

```
ClusterId: MkU3OEVBNTcwNTJENDM2Qk
LeaderId: 1
LeaderEpoch: 5
HighWatermark: 100
```

**Create a replicated topic and describe it:**

```
docker exec -it kafka1 kafka-topics.sh --create --topic secure-orders
\
  --bootstrap-server localhost:9092 --partitions 3 --replication-
factor 3

docker exec -it kafka1 kafka-topics.sh --describe --topic secure-
orders --bootstrap-server localhost:9092

Topic: secure-orders  PartitionCount: 3  ReplicationFactor: 3
  Partition: 0  Leader: 1  Replicas: 1,2,3  Isr: 1,2,3
  Partition: 1  Leader: 2  Replicas: 2,3,1  Isr: 2,3,1
  Partition: 2  Leader: 3  Replicas: 3,1,2  Isr: 3,1,2
```

**Simulate a broker failure (failover test):**

```
docker stop kafka3
docker exec -it kafka1 kafka-topics.sh --describe --topic secure-
orders --bootstrap-server localhost:9092

Partition: 2  Leader: 1  Replicas: 3,1,2  Isr: 1,2
```

Partition 2's Leader instantly moves to Broker 1; Broker 3 drops out of the ISR. Clients auto-refresh metadata and keep working.

**Force preferred-leader re-election once the broker returns:**

```
docker start kafka3
docker exec -it kafka1 kafka-leader-election.sh --bootstrap-server
localhost:9092 \
  --topic secure-orders --election-type PREFERRED --partitions 0,1,2
```

### 5.5 Inside the Controller (KRaft Mechanics)

On a broker crash: (1) the Controller detects the loss via heartbeat timeout, (2) removes the dead broker from ISR lists, (3) picks the first alive ISR member as new Leader, (4) persists the change to `__cluster_metadata` via Raft consensus, (5) sends `LeaderAndIsr` to affected brokers. Failover typically completes in ~5–10 seconds.

### 5.6 Critical Broker-Side Configuration Summary

| Config | Value | Why |
|---|---|---|
| `broker.id` | Unique int | Identity in the cluster |
| `replication.factor` | 3 | Survive one broker failure |
| `min.insync.replicas` | 2 | Forces `acks=all` producers |

| Config | Value | Why |
|---|---|---|
| | | to wait for 2 brokers |
| `unclean.leader.election.enable` | false | Prevents data loss from out-of-sync leader promotion |
| `default.replication.factor` | 3 | Applied to internal topics like `__consumer_offsets` |
| `controller.quorum.voters` | `1@kafka1:9093,...` | (KRaft) defines voting members |
| `rack.awareness` | AWS AZ IDs | Places replicas in different failure domains |

### 5.7 Java Client Connecting to the KRaft Cluster

```
Properties props = new Properties();
props.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG,
"localhost:9091,localhost:9092,localhost:9093");
props.put(ProducerConfig.ACKS_CONFIG, "all");
props.put(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, "true");
```

### 5.8 Full Java Cluster Toolkit

**`KafkaClusterAdmin.java` — inspect leaders/ISR/brokers programmatically:**

```
package com.kafka.admin;

import org.apache.kafka.clients.admin.AdminClient;
import org.apache.kafka.clients.admin.AdminClientConfig;
import org.apache.kafka.clients.admin.DescribeTopicsResult;
import org.apache.kafka.clients.admin.TopicDescription;
import org.apache.kafka.common.Node;
import org.apache.kafka.common.TopicPartitionInfo;
import java.util.Collections;
import java.util.Map;
import java.util.Properties;

public class KafkaClusterAdmin {
    private static final String BOOTSTRAP_SERVERS =
"localhost:9091,localhost:9092,localhost:9093";

    public static void main(String[] args) throws Exception {
        Properties props = new Properties();
        props.put(AdminClientConfig.BOOTSTRAP_SERVERS_CONFIG,
BOOTSTRAP_SERVERS);

        try (AdminClient admin = AdminClient.create(props)) {
            String topic = "secure-orders";
            DescribeTopicsResult result =
admin.describeTopics(Collections.singletonList(topic));
            Map<String, TopicDescription> descriptions =
```

```
result.allTopicNames().get();
            TopicDescription desc = descriptions.get(topic);

            System.out.println("Topic: " + topic + "  Partitions: " +
desc.partitions().size());
            for (TopicPartitionInfo p : desc.partitions()) {
                Node leader = p.leader();
                System.out.printf("  Partition %d -> Leader: %s (id=
%d)%n",
                        p.partition(), leader.host() + ":" +
leader.port(), leader.id());
                System.out.print("    Replicas: ");
                p.replicas().forEach(n -> System.out.print(n.id() + "
"));
                System.out.print("\n    ISR: ");
                p.isr().forEach(n -> System.out.print(n.id() + " "));
                System.out.println();
            }
            System.out.println("All brokers in cluster:");
            admin.describeCluster().nodes().get().forEach(node ->
                    System.out.printf("  Broker %d: %s:%d%n",
node.id(), node.host(), node.port()));
        }
    }
}
```

**`ClusterAwareProducer.java` — resilient producer with failover-aware config:**

```
package com.kafka.producer;

import org.apache.kafka.clients.producer.*;
import org.apache.kafka.common.serialization.StringSerializer;
import java.util.Properties;
import java.util.concurrent.atomic.AtomicInteger;

public class ClusterAwareProducer {
    private static final String BOOTSTRAP_SERVERS =
"localhost:9091,localhost:9092,localhost:9093";
    private static final String TOPIC = "secure-orders";

    public static void main(String[] args) {
        Properties props = new Properties();
        props.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG,
BOOTSTRAP_SERVERS);
        props.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG,
StringSerializer.class.getName());
        props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG,
StringSerializer.class.getName());
        props.put(ProducerConfig.ACKS_CONFIG, "all");
        props.put(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, "true");
```

```
props.put(ProducerConfig.MAX_IN_FLIGHT_REQUESTS_PER_CONNECTION, "5");
        props.put(ProducerConfig.RETRIES_CONFIG, Integer.MAX_VALUE);
        props.put(ProducerConfig.DELIVERY_TIMEOUT_MS_CONFIG, 120000);
        props.put(ProducerConfig.REQUEST_TIMEOUT_MS_CONFIG, 30000);
        props.put(ProducerConfig.COMPRESSION_TYPE_CONFIG, "snappy");
        props.put(ProducerConfig.METADATA_MAX_AGE_CONFIG, 5000); //
faster leader-change discovery

        try (KafkaProducer<String, String> producer = new
KafkaProducer<>(props)) {
            AtomicInteger success = new AtomicInteger(0), errors = new
AtomicInteger(0);
            for (int i = 1; i <= 20; i++) {
                String key = "user-" + (i % 5);
                String value = String.format("{\"orderId\":\"ord-%d\",
\"amount\":%d}", i, i * 100);
                producer.send(new ProducerRecord<>(TOPIC, key, value),
(metadata, exception) -> {
                    if (exception != null) {
                        errors.incrementAndGet();
                        System.err.printf("Send failed for key=%s: %s
%n", key, exception.getMessage());
                    } else {
                        success.incrementAndGet();
                        System.out.printf("Sent: key=%s, partition=%d,
offset=%d%n",
                                key, metadata.partition(),
metadata.offset());
                    }
                });
            }
            producer.flush();
            System.out.printf("Finished. Success: %d, Errors: %d%n",
success.get(), errors.get());
        }
    }
}
```

**`ClusterAwareConsumer.java` — manual commits + rebalance-aware, connected to all 3 brokers:**

```
package com.kafka.consumer;

import com.fasterxml.jackson.databind.ObjectMapper;
import org.apache.kafka.clients.consumer.*;
import org.apache.kafka.common.TopicPartition;
import org.apache.kafka.common.serialization.StringDeserializer;
import java.time.Duration;
import java.util.*;
```

```
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicLong;

public class ClusterAwareConsumer {
    private static final String BOOTSTRAP_SERVERS =
"localhost:9091,localhost:9092,localhost:9093";
    private static final String TOPIC = "secure-orders";
    private static final String GROUP_ID = "inventory-group";
    private static final ConcurrentHashMap<String, Boolean>
PROCESSED_STORE = new ConcurrentHashMap<>();
    private static final Map<TopicPartition, Long> LAST_COMMITTED =
new ConcurrentHashMap<>();
    private static final ObjectMapper objectMapper = new
ObjectMapper();

    static class Order {
        public String orderId; public int amount;
        public Order() {}
        @Override public String toString() { return "Order{id='" +
orderId + "', amount=" + amount + "}"; }
    }

    public static void main(String[] args) {
        Properties props = new Properties();
        props.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG,
BOOTSTRAP_SERVERS);
        props.put(ConsumerConfig.GROUP_ID_CONFIG, GROUP_ID);
        props.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG,
StringDeserializer.class.getName());
        props.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG,
StringDeserializer.class.getName());
        props.put(ConsumerConfig.ENABLE_AUTO_COMMIT_CONFIG, "false");
        props.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG,
"earliest");
        props.put(ConsumerConfig.MAX_POLL_RECORDS_CONFIG, "10");
        props.put(ConsumerConfig.ISOLATION_LEVEL_CONFIG,
"read_committed");
        props.put(ConsumerConfig.PARTITION_ASSIGNMENT_STRATEGY_CONFIG,

"org.apache.kafka.clients.consumer.CooperativeStickyAssignor");

        try (KafkaConsumer<String, String> consumer = new
KafkaConsumer<>(props)) {
            ConsumerRebalanceListener listener = new
ConsumerRebalanceListener() {
                @Override public void
onPartitionsAssigned(Collection<TopicPartition> partitions) {
                    System.out.println("Assigned: " + partitions);
                }
                @Override public void
```

```
onPartitionsRevoked(Collection<TopicPartition> partitions) {
                    System.out.println("Revoked: " + partitions + " -
committing final offsets...");
                    if (!LAST_COMMITTED.isEmpty()) {
                        Map<TopicPartition, OffsetAndMetadata>
commitMap = new HashMap<>();
                        LAST_COMMITTED.forEach((tp, offset) ->
commitMap.put(tp, new OffsetAndMetadata(offset)));
                        consumer.commitSync(commitMap);
                    }
                }
            };
            consumer.subscribe(Collections.singletonList(TOPIC),
listener);
            AtomicLong total = new AtomicLong(0);

            while (true) {
                ConsumerRecords<String, String> records =
consumer.poll(Duration.ofMillis(1000));
                if (records.isEmpty()) continue;
                Map<TopicPartition, Long> offsetsToCommit = new
HashMap<>();

                for (ConsumerRecord<String, String> record : records)
{
                    TopicPartition tp = new
TopicPartition(record.topic(), record.partition());
                    try {
                        Order order =
objectMapper.readValue(record.value(), Order.class);
                        if (PROCESSED_STORE.putIfAbsent(order.orderId,
true) == null) {
                            System.out.printf("Processed: partition=
%d, offset=%d, order=%s%n",
                                    record.partition(),
record.offset(), order);
                            total.incrementAndGet();
                        } else {
                            System.out.printf("Duplicate skipped:
partition=%d, offset=%d, id=%s%n",
                                    record.partition(),
record.offset(), order.orderId);
                        }
                        offsetsToCommit.put(tp, record.offset());
                    } catch (Exception e) {
                        System.err.printf("Poison pill at partition=
%d, offset=%d%n", record.partition(), record.offset());
                        offsetsToCommit.put(tp, record.offset());
                    }
                }
```

```
                if (!offsetsToCommit.isEmpty()) {
                    Map<TopicPartition, OffsetAndMetadata> commitMap =
new HashMap<>();
                    offsetsToCommit.forEach((tp, offset) -> {
                        long next = offset + 1;
                        commitMap.put(tp, new
OffsetAndMetadata(next));
                        LAST_COMMITTED.put(tp, next);
                    });
                    consumer.commitSync(commitMap);
                }
                System.out.printf("Total processed so far: %d%n",
total.get());
            }
        }
    }
}
```

### 5.9 Failover Validation Script

```
#!/bin/bash
echo "Describing initial topic..."
docker exec kafka1 kafka-topics.sh --describe --topic secure-orders --
bootstrap-server localhost:9092

echo "Killing Broker 3 (leader for Partition 2)..."
docker stop kafka3
sleep 15

echo "Describing topic after failure..."
docker exec kafka1 kafka-topics.sh --describe --topic secure-orders --
bootstrap-server localhost:9092

echo "Producing a test message via Broker 1..."
echo "TestFailover" | docker exec -i kafka1 kafka-console-producer.sh
--topic secure-orders --bootstrap-server localhost:9092

echo "Consuming the test message..."
docker exec kafka1 kafka-console-consumer.sh --topic secure-orders --
bootstrap-server localhost:9092 --from-beginning --max-messages 1
```

**Expected result:** the Producer succeeds against the newly-elected Leader and the Consumer reads the message — zero data loss and zero downtime despite a broker being terminated.

---

## Chapter 6 — Consolidated Java Producer & Consumer Code

This chapter distills the producer/consumer mechanics from Chapters 3–5 into a single, copy-pasteable codebase, highlighting **callback handling**, **message keys**, and **cooperative rebalancing**.

### 6.0 Project Setup

```
<dependencies>
    <dependency>
        <groupId>org.apache.kafka</groupId>
        <artifactId>kafka-clients</artifactId>
        <version>3.6.0</version>
    </dependency>
    <dependency>
        <groupId>com.fasterxml.jackson.core</groupId>
        <artifactId>jackson-databind</artifactId>
        <version>2.15.2</version>
    </dependency>
    <dependency>
        <groupId>org.slf4j</groupId>
        <artifactId>slf4j-simple</artifactId>
        <version>2.0.9</version>
    </dependency>
</dependencies>
```

### 6.1 Order.java

```
package com.kafka.model;

public class Order {
    private String orderId;
    private int amount;
    public Order() {}
    public Order(String orderId, int amount) { this.orderId = orderId;
this.amount = amount; }
    public String getOrderId() { return orderId; }
    public void setOrderId(String orderId) { this.orderId = orderId; }
    public int getAmount() { return amount; }
    public void setAmount(int amount) { this.amount = amount; }
    @Override public String toString() { return "Order{id='" + orderId
+ "', amount=" + amount + "}"; }
}
```

### 6.2 ProducerDemo.java

```
package com.kafka.producer;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.kafka.model.Order;
import org.apache.kafka.clients.producer.*;
import org.apache.kafka.common.serialization.StringSerializer;
import java.util.Properties;
```

```java
import java.util.concurrent.atomic.AtomicInteger;

public class ProducerDemo {
    private static final String BOOTSTRAP_SERVERS = "localhost:9092";
    private static final String TOPIC = "orders";

    public static void main(String[] args) throws Exception {
        Properties props = new Properties();
        props.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG,
BOOTSTRAP_SERVERS);
        props.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG,
StringSerializer.class.getName());
        props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG,
StringSerializer.class.getName());

        props.put(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, "true");
// forces acks=all, retries=MAX
        props.put(ProducerConfig.ACKS_CONFIG, "all");

props.put(ProducerConfig.MAX_IN_FLIGHT_REQUESTS_PER_CONNECTION, "5");
        props.put(ProducerConfig.RETRIES_CONFIG, Integer.MAX_VALUE);
        props.put(ProducerConfig.DELIVERY_TIMEOUT_MS_CONFIG, 120000);

        props.put(ProducerConfig.COMPRESSION_TYPE_CONFIG, "snappy");
        props.put(ProducerConfig.BATCH_SIZE_CONFIG, 16384);
        props.put(ProducerConfig.LINGER_MS_CONFIG, 10);

        try (KafkaProducer<String, String> producer = new
KafkaProducer<>(props)) {
            ObjectMapper mapper = new ObjectMapper();
            AtomicInteger successCount = new AtomicInteger(0),
errorCount = new AtomicInteger(0);

            for (int i = 1; i <= 20; i++) {
                // KEY: "user_123" -> ALL messages go to the SAME
partition (ordering guarantee)
                String key = "user_123";
                Order order = new Order("order-" + i, i * 100);
                String jsonValue = mapper.writeValueAsString(order);
                ProducerRecord<String, String> record = new
ProducerRecord<>(TOPIC, key, jsonValue);

                // ASYNCHRONOUS SEND WITH CALLBACK HANDLING
                producer.send(record, (metadata, exception) -> {
                    if (exception != null) {
                        errorCount.incrementAndGet();
                        System.err.printf("Send FAILED for key=%s,
value=%s : %s%n", key, jsonValue, exception.getMessage());
                    } else {
                        successCount.incrementAndGet();
```

```
                        System.out.printf("Sent: key=%s, partition=%d,
offset=%d, timestamp=%d%n",
                                          key, metadata.partition(),
metadata.offset(), metadata.timestamp());
                    }
                });
                Thread.sleep(50);
            }

            producer.flush();
            System.out.printf("Producer finished. Success: %d, Errors:
%d%n", successCount.get(), errorCount.get());
        }
    }
}
```

### 6.3 ConsumerDemo.java

```
package com.kafka.consumer;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.kafka.model.Order;
import org.apache.kafka.clients.consumer.*;
import org.apache.kafka.common.TopicPartition;
import org.apache.kafka.common.serialization.StringDeserializer;
import java.time.Duration;
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;

public class ConsumerDemo {
    private static final String BOOTSTRAP_SERVERS = "localhost:9092";
    private static final String TOPIC = "orders";
    private static final String GROUP_ID = "inventory-group";
    private static final ConcurrentHashMap<String, Boolean>
PROCESSED_STORE = new ConcurrentHashMap<>();

    public static void main(String[] args) {
        Properties props = new Properties();
        props.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG,
BOOTSTRAP_SERVERS);
        props.put(ConsumerConfig.GROUP_ID_CONFIG, GROUP_ID);
        props.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG,
StringDeserializer.class.getName());
        props.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG,
StringDeserializer.class.getName());

        props.put(ConsumerConfig.ENABLE_AUTO_COMMIT_CONFIG, "false");
// manual commit
        props.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG,
"earliest");
        props.put(ConsumerConfig.MAX_POLL_RECORDS_CONFIG, "10");
```

```
        props.put(ConsumerConfig.ISOLATION_LEVEL_CONFIG,
"read_committed");

        // COOPERATIVE REBALANCING - minimizes stop-the-world downtime
        props.put(ConsumerConfig.PARTITION_ASSIGNMENT_STRATEGY_CONFIG,

"org.apache.kafka.clients.consumer.CooperativeStickyAssignor");

        try (KafkaConsumer<String, String> consumer = new
KafkaConsumer<>(props)) {
            final Map<TopicPartition, Long> currentOffsets = new
ConcurrentHashMap<>();

            ConsumerRebalanceListener rebalanceListener = new
ConsumerRebalanceListener() {
                @Override public void
onPartitionsAssigned(Collection<TopicPartition> partitions) {
                    System.out.println("Partitions Assigned
(Cooperative): " + partitions);
                }
                @Override public void
onPartitionsRevoked(Collection<TopicPartition> partitions) {
                    System.out.println("Partitions Revoked
(Cooperative): " + partitions);
                    if (!currentOffsets.isEmpty()) {
                        Map<TopicPartition, OffsetAndMetadata>
commitMap = new HashMap<>();
                        currentOffsets.forEach((tp, offset) ->
commitMap.put(tp, new OffsetAndMetadata(offset + 1)));
                        consumer.commitSync(commitMap);
                        System.out.printf("Committed final offsets: %s
%n", commitMap);
                    }
                }
            };

            consumer.subscribe(Collections.singletonList(TOPIC),
rebalanceListener);
            System.out.println("Consumer started. Group: " +
GROUP_ID);
            ObjectMapper mapper = new ObjectMapper();

            while (true) {
                ConsumerRecords<String, String> records =
consumer.poll(Duration.ofMillis(1000));
                if (records.isEmpty()) continue;

                Map<TopicPartition, Long> offsetsToCommit = new
HashMap<>();
                for (ConsumerRecord<String, String> record : records)
```

```
{
                    TopicPartition tp = new
TopicPartition(record.topic(), record.partition());
                    try {
                        Order order = mapper.readValue(record.value(),
Order.class);
                        if
(PROCESSED_STORE.putIfAbsent(order.getOrderId(), true) == null) {
                            System.out.printf("PROCESSED: partition=
%d, offset=%d, order=%s%n",
                                    record.partition(),
record.offset(), order);
                        } else {
                            System.out.printf("DUPLICATE SKIPPED:
partition=%d, offset=%d, id=%s%n",
                                    record.partition(),
record.offset(), order.getOrderId());
                        }
                        offsetsToCommit.put(tp, record.offset());
                    } catch (Exception e) {
                        System.err.printf("POISON PILL! Partition: %d,
Offset: %d, RawValue: %s%n",
                                record.partition(), record.offset(),
record.value());
                        offsetsToCommit.put(tp, record.offset());
                    }
                }

                if (!offsetsToCommit.isEmpty()) {
                    Map<TopicPartition, OffsetAndMetadata> commitMap =
new HashMap<>();
                    offsetsToCommit.forEach((tp, offset) -> {
                        commitMap.put(tp, new OffsetAndMetadata(offset
+ 1));
                        currentOffsets.put(tp, offset);
                    });
                    consumer.commitSync(commitMap);
                    System.out.printf("Committed offsets: %s%n",
commitMap);
                }
            }
        }
    }
}
```

### 6.4 Key Features Highlighted

| Feature | Where implemented | Why it matters |
|---|---|---|
| Producer Callback | `producer.send(record, (metadata, ex) -> {...})` | Async success/failure handling without blocking |

| Feature | Where implemented | Why it matters |
|---|---|---|
| | | the main thread |
| Message Keys | `new ProducerRecord<>(TOPIC, key, jsonValue)` | Same key -> same partition -> strict per-key ordering |
| Cooperative Rebalancing | `CooperativeStickyAssignor` | Incremental revoke/assign instead of all-at-once, reducing pause time |
| Rebalance Listener | `onPartitionsRevoked` + `commitSync` | Commits the exact offset before losing a partition, limiting duplication |
| Manual Offsets | `enable.auto.commit=false` + `commitSync` in the loop | Atomic "process then commit" semantics |
| Poison Pill Resilience | `catch (Exception e)` + still advancing the offset | One bad message can't crash the whole consumer |

### 6.5 Running the Demo

```
docker-compose -f docker-compose-kraft.yml up -d
docker exec -it kafka1 kafka-topics.sh --create --topic orders \
  --bootstrap-server localhost:9092 --partitions 3 --replication-
factor 1

mvn clean compile
mvn exec:java -Dexec.mainClass="com.kafka.consumer.ConsumerDemo"     #
Terminal 1
mvn exec:java -Dexec.mainClass="com.kafka.producer.ProducerDemo"     #
Terminal 2
```

---

## Chapter 7 — Offset Management & Delivery Semantics

### 7.1 Definition (The 3 Delivery Patterns)

Everything hinges on the ordering of **Processing** vs. **Committing**:

- **At-Most-Once** (commit BEFORE processing): if processing crashes, the offset is already advanced -> the message is lost forever. Use for max-throughput, loss-tolerant data.
- **At-Least-Once** (process THEN commit): if the commit itself crashes after a successful process, the consumer re-reads the same message on restart -> duplicates, but zero data loss.
- **Exactly-Once** (At-Least-Once + Idempotent Sink): duplicates may still be re-processed, but the downstream store (DB, Streams state) recognizes and neutralizes them.

## 7.2 Architectural Flow Diagram — The Reprocess-on-Crash Loop




*Chapter 7 - The Reprocess-on-Crash Loop*

*Figure: Chapter 7 - The Reprocess-on-Crash Loop*

**The critical rule:** commit-before-process = At-Most-Once; commit-after-process = At-Least-Once.

## 7.3 Hands-On: End-to-End Java Solution (Simulating a Crash)

`OffsetManagementDemo.java` demonstrates all three patterns via a configurable flag, plus a deliberate crash trigger to prove the reprocessing behavior.

```java
package com.kafka.consumer;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.kafka.model.Order;
import org.apache.kafka.clients.consumer.*;
import org.apache.kafka.common.TopicPartition;
import org.apache.kafka.common.serialization.StringDeserializer;
import java.time.Duration;
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicInteger;

public class OffsetManagementDemo {
    private static final String BOOTSTRAP_SERVERS = "localhost:9092";
    private static final String TOPIC = "orders";
    private static final String GROUP_ID = "offset-demo-group";

    public enum DeliverySemantic { AT_MOST_ONCE, AT_LEAST_ONCE,
EXACTLY_ONCE }
    private static final DeliverySemantic MODE =
DeliverySemantic.AT_LEAST_ONCE;

    private static final ConcurrentHashMap<String, Boolean>
IDEMPOTENT_STORE = new ConcurrentHashMap<>();
    private static final AtomicInteger processedCounter = new
AtomicInteger(0);
    private static final int CRASH_AFTER_RECORDS = 3;

    public static void main(String[] args) throws Exception {
        Properties props = new Properties();
        props.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG,
BOOTSTRAP_SERVERS);
        props.put(ConsumerConfig.GROUP_ID_CONFIG, GROUP_ID);
        props.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG,
StringDeserializer.class.getName());
        props.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG,
StringDeserializer.class.getName());
        props.put(ConsumerConfig.ENABLE_AUTO_COMMIT_CONFIG, "false");
        props.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG,
"earliest");
        props.put(ConsumerConfig.ISOLATION_LEVEL_CONFIG,
"read_committed");

        try (KafkaConsumer<String, String> consumer = new
KafkaConsumer<>(props)) {
            consumer.subscribe(Collections.singletonList(TOPIC));
            ObjectMapper mapper = new ObjectMapper();
            System.out.println("Starting Consumer in mode: " + MODE);

            while (true) {
```

```
                ConsumerRecords<String, String> records =
consumer.poll(Duration.ofMillis(1000));
                if (records.isEmpty()) continue;

                if (MODE == DeliverySemantic.AT_MOST_ONCE) {
                    consumer.commitSync(); // commit BEFORE processing
                    System.out.println("[At-Most-Once] Committed
offsets BEFORE processing.");
                }

                Map<TopicPartition, Long> offsetsToCommit = new
HashMap<>();
                for (ConsumerRecord<String, String> record : records)
{
                    try {
                        Order order = mapper.readValue(record.value(),
Order.class);

                        if (MODE == DeliverySemantic.EXACTLY_ONCE) {
                            if
(IDEMPOTENT_STORE.putIfAbsent(order.getOrderId(), true) != null) {
                                System.out.printf("[Exactly-Once]
Skipping duplicate: %s%n", order.getOrderId());
                                offsetsToCommit.put(new
TopicPartition(record.topic(), record.partition()), record.offset());
                                continue;
                            }
                        }

                        System.out.printf("PROCESSING: Partition=%d,
Offset=%d, Order=%s%n",
                                record.partition(), record.offset(),
order);
                        Thread.sleep(100);

                        int currentCount =
processedCounter.incrementAndGet();
                        if (currentCount == CRASH_AFTER_RECORDS) {
                            System.err.println("SIMULATED CRASH!
Processing stopped at record " + currentCount);
                            throw new RuntimeException("Intentional
crash to test offset management!");
                        }

                        offsetsToCommit.put(new
TopicPartition(record.topic(), record.partition()), record.offset());
                    } catch (Exception e) {
                        System.err.println("Error processing record: "
+ record.value());
                        offsetsToCommit.put(new
```

```
TopicPartition(record.topic(), record.partition()), record.offset());
                    }
                }

                if (MODE == DeliverySemantic.AT_LEAST_ONCE || MODE ==
DeliverySemantic.EXACTLY_ONCE) {
                    if (!offsetsToCommit.isEmpty()) {
                        Map<TopicPartition, OffsetAndMetadata>
commitMap = new HashMap<>();
                        offsetsToCommit.forEach((tp, offset) ->
commitMap.put(tp, new OffsetAndMetadata(offset + 1)));
                        consumer.commitSync(commitMap);
                        System.out.println("[At-Least/Exactly-Once]
Committed offsets AFTER processing: " + commitMap);
                    }
                }
            }
        } catch (Exception e) {
            System.err.println("Consumer crashed due to: " +
e.getMessage());
        }
    }
}
```

### 7.4 Running the Simulation

1. Produce 10 orders with `ProducerDemo` (Chapter 6).
2. Run with `MODE = AT_LEAST_ONCE`: it processes orders 1–3, then simulates a crash at order 3.
3. Restart: the consumer **re-reads order 3** because its offset was never committed — proving At-Least-Once duplication.
4. Switch to `AT_MOST_ONCE`, reset the group offset, rerun: the offset commits *before* processing, so a crash at order 3 means **order 3 is permanently skipped** on restart — proving data loss.
5. Switch to `EXACTLY_ONCE`, run it twice: the second run re-reads order 3, but `IDEMPOTENT_STORE` already has it, so it's logged as a duplicate and skipped at the application layer — the final state is as if it was processed exactly once.

### 7.5 Real-Time Usage

| Scenario | Required Semantic | Why |
|---|---|---|
| IoT Sensor Metrics | At-Most-Once | Losing 1 reading of 10,000/sec is irrelevant; throughput is king |
| Email Notification Service | At-Least-Once | Must eventually send; a duplicate is safer than a missed email |

| Scenario | Required Semantic | Why |
|---|---|---|
| Bank Account Debit/Credit | Exactly-Once | Must debit exactly once; `transaction_id` PK constraint naturally dedupes replays |
| Kafka to HDFS (Batch) | At-Least-Once | Unique partition paths + compaction step dedupe downstream |

### 7.6 Offset Reset Commands (The Escape Hatch)

```
# Describe current offsets and lag
bin/kafka-consumer-groups.sh --describe --group offset-demo-group --
bootstrap-server localhost:9092

# Reset to earliest (replay everything)
bin/kafka-consumer-groups.sh --group offset-demo-group --bootstrap-
server localhost:9092 --to-earliest --execute

# Reset to latest (skip all current data)
bin/kafka-consumer-groups.sh --group offset-demo-group --bootstrap-
server localhost:9092 --to-latest --execute

# Reset a specific partition to a specific offset (surgical repair)
bin/kafka-consumer-groups.sh --group offset-demo-group --bootstrap-
server localhost:9092 \
  --to-offset 10 --topic orders:0 --execute

# Reset by timestamp (reprocess data from a point in time)
bin/kafka-consumer-groups.sh --group offset-demo-group --bootstrap-
server localhost:9092 \
  --to-datetime 2026-07-12T10:00:00.000 --execute

# Preview any of the above without applying it
# (add --dry-run to any command above)
```

### 7.7 The "Commit Failure" Edge Case

If `commitSync()` itself throws (e.g. `TimeoutException`), the offset is **not** advanced in `__consumer_offsets`, so the next `poll()` will legitimately re-deliver the same batch — this is safe for At-Least-Once but can loop forever if commits never succeed (e.g. broker down). Mitigation: `commitAsync()` with a callback and bounded retry/backoff, understanding that async commits can be reordered.

> **The Golden Rule of Offsets:** Never commit an offset for a message you haven't successfully processed (unless using At-Most-Once). Always commit `offset + 1` so you don't re-read the message you just handled.

### 7.8 Summary Checklist

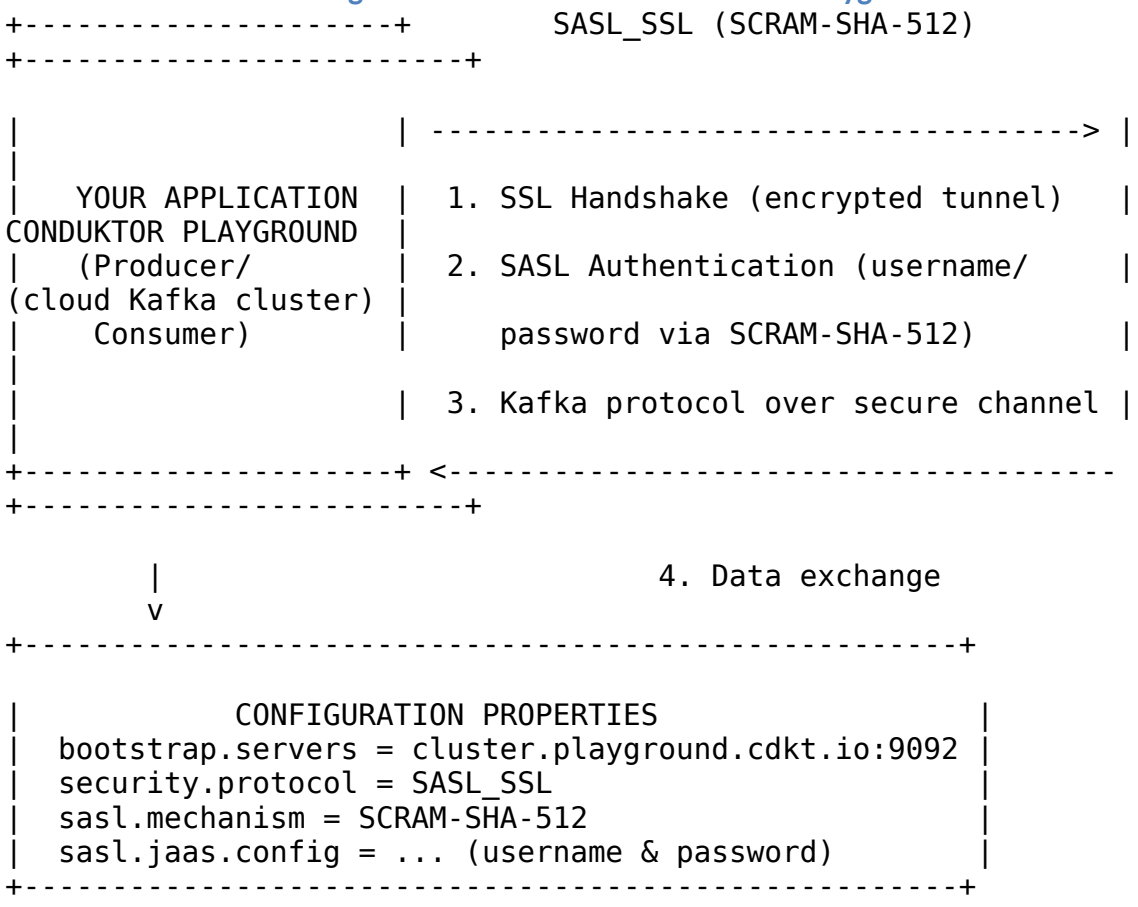
| Pattern | Commit Timing | Duplicates? | Data Loss? | Idempotency Required? |
|---|---|---|---|---|
| At-Most-Once | Before Process | No | Yes | No |
| At-Least-Once | After Process | Yes | No | Yes (to avoid downstream duplicates) |
| Exactly-Once | After Process + Idempotent Sink | No (masked) | No | Yes (critical) |

## Chapter 8 — Cloud Setup (Conduktor Playground)

### 8.1 Definition

- **Conduktor Playground:** A free, cloud-hosted Kafka cluster for learning/experimentation — no local cluster setup required.
- **SASL_SSL:** A security protocol combining SASL (authentication) with SSL/TLS (encryption). The Playground uses **SCRAM-SHA-512** as its SASL mechanism.

### 8.2 Architectural Flow Diagram — Secure Connection to the Playground



1. An SSL handshake establishes an encrypted tunnel.
2. The client authenticates over that tunnel using SCRAM-SHA-512 (username/password).
3. Once authenticated, ordinary Kafka protocol requests (produce/fetch) flow over the secure channel.
4. Data is exchanged exactly as with any Kafka cluster — the only difference is the security-layer wrapping.

### 8.3 The Exact Properties Block

```
properties.setProperty("bootstrap.servers",
"cluster.playground.cdkt.io:9092");
properties.setProperty("security.protocol", "SASL_SSL");
properties.setProperty("sasl.mechanism", "SCRAM-SHA-512");
properties.setProperty("sasl.jaas.config",
    "org.apache.kafka.common.security.scram.ScramLoginModule required
" +
    "username='<your-username>' " +
    "password='<your-password>';");
```

Replace `<your-username>` / `<your-password>` with your Playground credentials. Never hardcode these in source — use environment variables or a secure vault.

### 8.4 Hands-On: `ProducerDemo` for Conduktor Playground

```
package io.conduktor.demos.kafka;

import org.apache.kafka.clients.producer.KafkaProducer;
import org.apache.kafka.clients.producer.ProducerRecord;
import org.apache.kafka.common.serialization.StringSerializer;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import java.util.Properties;

public class ProducerDemo {
    private static final Logger log =
LoggerFactory.getLogger(ProducerDemo.class);

    public static void main(String[] args) {
        log.info("Starting Producer...");

        Properties properties = new Properties();
        properties.setProperty("bootstrap.servers",
"cluster.playground.cdkt.io:9092");
        properties.setProperty("security.protocol", "SASL_SSL");
        properties.setProperty("sasl.mechanism", "SCRAM-SHA-512");
        properties.setProperty("sasl.jaas.config",

"org.apache.kafka.common.security.scram.ScramLoginModule required " +
                "username='<your-username>' " +
```

```
                "password='<your-password>';");

        properties.setProperty("key.serializer",
StringSerializer.class.getName());
        properties.setProperty("value.serializer",
StringSerializer.class.getName());

        KafkaProducer<String, String> producer = new
KafkaProducer<>(properties);
        ProducerRecord<String, String> producerRecord = new
ProducerRecord<>("demo_java", "hello, world!");
        producer.send(producerRecord);
        producer.flush();
        producer.close();
    }
}
```

### 8.5 ConsumerDemo for Conduktor Playground

```
package io.conduktor.demos.kafka;

import org.apache.kafka.clients.consumer.ConsumerRecord;
import org.apache.kafka.clients.consumer.ConsumerRecords;
import org.apache.kafka.clients.consumer.KafkaConsumer;
import org.apache.kafka.common.serialization.StringDeserializer;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import java.time.Duration;
import java.util.Arrays;
import java.util.Properties;

public class ConsumerDemo {
    private static final Logger log =
LoggerFactory.getLogger(ConsumerDemo.class);

    public static void main(String[] args) {
        log.info("Starting Consumer...");

        Properties properties = new Properties();
        properties.setProperty("bootstrap.servers",
"cluster.playground.cdkt.io:9092");
        properties.setProperty("security.protocol", "SASL_SSL");
        properties.setProperty("sasl.mechanism", "SCRAM-SHA-512");
        properties.setProperty("sasl.jaas.config",

"org.apache.kafka.common.security.scram.ScramLoginModule required " +
                "username='<your-username>' " +
                "password='<your-password>';");

        properties.setProperty("key.deserializer",
StringDeserializer.class.getName());
```

```
        properties.setProperty("value.deserializer",
StringDeserializer.class.getName());
        properties.setProperty("group.id", "my-java-application");
        properties.setProperty("auto.offset.reset", "earliest");

        KafkaConsumer<String, String> consumer = new
KafkaConsumer<>(properties);
        consumer.subscribe(Arrays.asList("demo_java"));

        while (true) {
            ConsumerRecords<String, String> records =
consumer.poll(Duration.ofMillis(1000));
            for (ConsumerRecord<String, String> record : records) {
                log.info("Key: " + record.key() + ", Value: " +
record.value());
                log.info("Partition: " + record.partition() + ",
Offset: " + record.offset());
            }
        }
    }
}
```

### 8.6 Real-Time Usage

- **Learning/Prototyping:** zero-overhead environment for beginners to test producers/consumers.
- **Demo Environments:** quick, presentable demonstrations for clients/stakeholders.
- **Connectivity Testing:** validate a SASL_SSL configuration before pointing at production.

### 8.7 Conduktor Console Configuration

```
security.protocol=SASL_SSL
sasl.mechanism=SCRAM-SHA-512
sasl.jaas.config=org.apache.kafka.common.security.scram.ScramLoginModu
le required username='<username>' password='<password>';
```

This lets you visually inspect topics, produce/consume messages, and manage the cluster from a UI using the same credentials as your Java client.

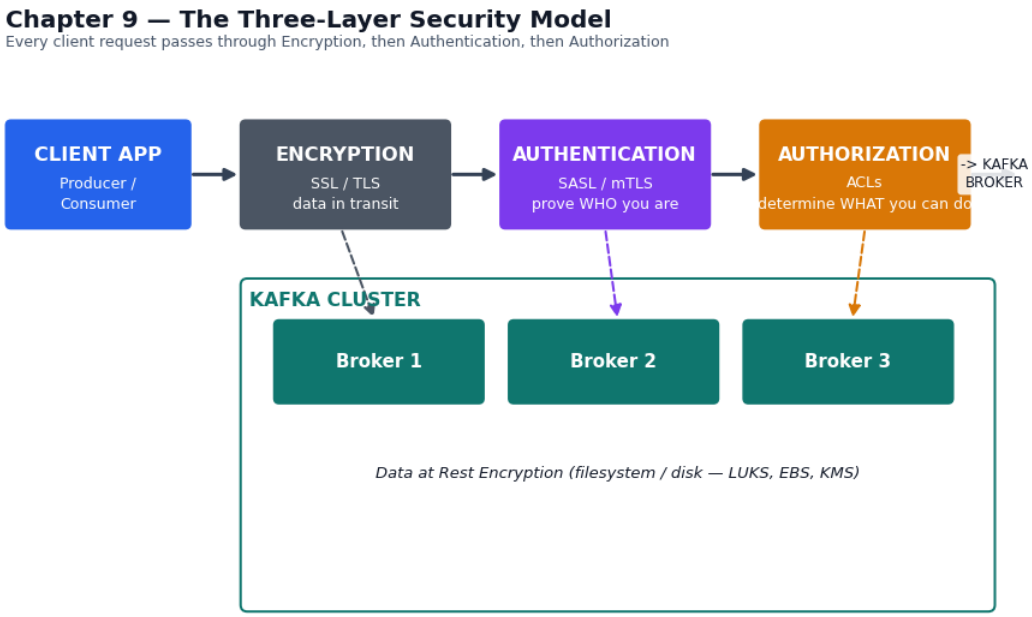
### 8.8 Key Takeaways

- SASL_SSL is the standard for secure cloud connections — authentication + encryption together.
- The Conduktor Playground offers a free, hassle-free way to practice against a real cluster.
- The four properties you always need: `bootstrap.servers`, `security.protocol`, `sasl.mechanism`, `sasl.jaas.config`.
- Always secure credentials via environment variables or a secret manager, never hardcoded.

## Chapter 9 — Kafka Security Management

Security in Apache Kafka is a layered defense strategy built on three core pillars: **Encryption** (data in transit/at rest), **Authentication** (verifying identity), and **Authorization** (controlling access) — plus cluster hardening, auditing, and key rotation.

### 9.1 Architectural Flow Diagram — The Three-Layer Security Model




*Chapter 9 - The Three-Layer Security Model*

*Figure: Chapter 9 - The Three-Layer Security Model*

**The flow, step by step:** (1) Encryption establishes a scrambled, eavesdropper-proof tunnel. (2) Authentication makes the client prove its identity over that tunnel. (3) Authorization then checks the now-known identity against ACLs to decide if the specific operation is permitted.

### 9.2 Encryption (Data in Transit & At Rest)

- **In Transit (SSL/TLS):** encrypts client-broker and inter-broker traffic (still called "SSL" for historical reasons).
- **At Rest:** Kafka doesn't natively encrypt partition files — use filesystem-level encryption (LUKS) or cloud disk encryption (AWS EBS/KMS), or application-level field encryption for PII.

**Generate a Keystore & Truststore:**

```
keytool -keystore kafka.server.keystore.jks -alias localhost -validity
365 -genkey -keyalg RSA
openssl req -new -x509 -keyout ca-key -out ca-cert -days 365
keytool -keystore kafka.server.keystore.jks -alias localhost -certreq
-file cert-file
openssl x509 -req -CA ca-cert -CAkey ca-key -in cert-file -out cert-
signed -days 365 -CAcreateserial
keytool -keystore kafka.server.truststore.jks -alias CARoot -import -
file ca-cert
keytool -keystore kafka.server.keystore.jks -alias localhost -import -
file cert-signed
```

**Broker config (`server.properties`):**

```
listeners=PLAINTEXT://:9092,SSL://:9093
advertised.listeners=PLAINTEXT://localhost:9092,SSL://localhost:9093
ssl.keystore.location=/path/to/kafka.server.keystore.jks
ssl.keystore.password=keystore-password
ssl.key.password=key-password
ssl.truststore.location=/path/to/kafka.server.truststore.jks
ssl.truststore.password=truststore-password
ssl.client.auth=required
```

**Java client (SSL):**

```
Properties props = new Properties();
props.put("bootstrap.servers", "localhost:9093");
props.put("security.protocol", "SSL");
props.put("ssl.truststore.location",
"/path/to/client.truststore.jks");
props.put("ssl.truststore.password", "truststore-password");
props.put("ssl.keystore.location", "/path/to/client.keystore.jks");
props.put("ssl.keystore.password", "keystore-password");
props.put("ssl.key.password", "key-password");
KafkaProducer<String, String> producer = new KafkaProducer<>(props);
```

**Real-Time Usage:** PCI-DSS/GDPR compliance for all inter-broker/client traffic; TLS securing multi-cloud/region data transfer; LUKS/KMS disk encryption for healthcare PII.

### 9.3 Authentication (Proving Identity)

| Mechanism | Description | Use Case |
|---|---|---|
| mTLS | Client presents a signed cert; broker verifies against a CA | High-security PKI environments |
| SASL/PLAIN | Username/password over TLS | Simple, only safe with TLS |
| SASL/SCRAM | Salted challenge-response (SHA-256/512); no plaintext passwords | Recommended default; supports rotation |
| SASL/GSSAPI (Kerberos) | Enterprise SSO integration | Large orgs with existing |

| Mechanism | Description | Use Case |
|---|---|---|
| | | AD/Kerberos |
| SASL/OAUTHBEARER | OAuth 2.0 / OIDC tokens | Modern cloud-native architectures |

**Broker config (SCRAM-SHA-512):**

```
listeners=SASL_SSL://:9093
advertised.listeners=SASL_SSL://localhost:9093
security.protocol=SASL_SSL
sasl.enabled.mechanisms=SCRAM-SHA-512
sasl.mechanism.inter.broker.protocol=SCRAM-SHA-512
listener.name.sasl_ssl.scram-sha-
512.sasl.jaas.config=org.apache.kafka.common.security.scram.ScramLogin
Module required;
```

**Create SCRAM users:**

```
bin/kafka-configs.sh --bootstrap-server localhost:9093 --alter \
  --add-config 'SCRAM-SHA-512=[password=producer-secret]' \
  --entity-type users --entity-name producer-user

bin/kafka-configs.sh --bootstrap-server localhost:9093 --alter \
  --add-config 'SCRAM-SHA-512=[password=consumer-secret]' \
  --entity-type users --entity-name consumer-user
```

**Java client (SASL/SCRAM):**

```
Properties props = new Properties();
props.put("bootstrap.servers", "localhost:9093");
props.put("security.protocol", "SASL_SSL");
props.put("sasl.mechanism", "SCRAM-SHA-512");
props.put("sasl.jaas.config",
    "org.apache.kafka.common.security.scram.ScramLoginModule required
" +
    "username='producer-user' password='producer-secret';");
props.put("ssl.truststore.location",
"/path/to/client.truststore.jks");
props.put("ssl.truststore.password", "truststore-password");
KafkaProducer<String, String> producer = new KafkaProducer<>(props);
```

**Java client (SASL/OAUTHBEARER):**

```
Properties props = new Properties();
props.put("bootstrap.servers", "localhost:9093");
props.put("security.protocol", "SASL_SSL");
props.put("sasl.mechanism", "OAUTHBEARER");
props.put("sasl.jaas.config",

"org.apache.kafka.common.security.oauthbearer.OAuthBearerLoginModule
required " +
```

```
    "clientId='my-client' " +
    "clientSecret='my-secret' " +
    "tokenEndpointUrl='https://auth.my-company.com/oauth2/token' " +
    "scope='kafka';");
props.put("ssl.truststore.location",
"/path/to/client.truststore.jks");
props.put("ssl.truststore.password", "truststore-password");
```

**Real-Time Usage:** SASL/SCRAM for e-commerce microservices; Kerberos for banks with existing Active Directory; OAuth 2.0 for SaaS companies using Okta/Auth0.

### 9.4 Authorization (Access Control)

An ACL states: *"Principal P is [Allowed/Denied] Operation O from Host H on Resource R."* Operations include Read, Write, Create, Delete, Describe, Alter, All. Resources include Topic, Group, Cluster, TransactionalId.

**Enable the authorizer (`server.properties`):**

```
authorizer.class.name=kafka.security.authorizer.AclAuthorizer
allow.everyone.if.no.acl.found=false
```

**Add ACLs:**

```
bin/kafka-acls.sh --bootstrap-server localhost:9093 \
  --add --allow-principal User:producer-user --operation Write --topic
orders

bin/kafka-acls.sh --bootstrap-server localhost:9093 \
  --add --allow-principal User:consumer-user \
  --operation Read --topic orders \
  --operation Read --group inventory-group

bin/kafka-acls.sh --bootstrap-server localhost:9093 \
  --add --deny-principal User:unauthorized-user --operation Describe
--cluster

bin/kafka-acls.sh --bootstrap-server localhost:9093 --list
```

The client code is unchanged from the authentication examples — ACLs are enforced broker-side. If `producer-user` (WRITE only) attempts to read `orders`, the broker rejects the request with *"Not authorized to access topics: [orders]"*.

**Real-Time Usage:** multi-tenant SaaS with per-tenant ACL scoping; governance (finance teams Read-only, CI/CD Alter-only); SOC2/ISO27001 least-privilege enforcement.

### 9.5 Advanced Cluster & Ecosystem Security

**KRaft controller security:**

```
controller.quorum.voters=1@localhost:9093,2@localhost:9094,3@localhost
:9095
```

```
controller.quorum.auth.enable=true
security.inter.broker.protocol=SASL_SSL
sasl.mechanism.inter.broker.protocol=SCRAM-SHA-512
```

**Kafka Connect security:**

```
rest.extension.classes=io.confluent.connect.security.ConnectSecurityEx
tension
security.protocol=SASL_SSL
sasl.mechanism=SCRAM-SHA-512
```

**Schema Registry security:**

```
kafkastore.security.protocol=SASL_SSL
kafkastore.sasl.mechanism=SCRAM-SHA-512
```

**Network segmentation — separate internal/external listeners:**

```
listeners=INTERNAL://:9091,EXTERNAL://:9093
listener.security.protocol.map=INTERNAL:PLAINTEXT,EXTERNAL:SASL_SSL
advertised.listeners=INTERNAL://broker1.internal:9091,EXTERNAL://kafka
.mycompany.com:9093
```

### 9.6 Auditing and Monitoring

**Enable audit logging:**

```
log4j.logger.kafka.authorizer.logger=DEBUG, authorizerAppender
log4j.appender.authorizerAppender=org.apache.log4j.DailyRollingFileApp
ender
log4j.appender.authorizerAppender.File=/var/log/kafka/kafka-
authorizer.log
```

Captures: successful/failed authentication attempts, authorization denials, and ACL changes.

**Certificate rotation (rolling, zero downtime):**

```
keytool -genkey -alias localhost -keyalg RSA -keystore
kafka.server.keystore.jks -validity 365
# Roll brokers one at a time: update keystore, restart, repeat per
broker
```

**SCRAM credential rotation:**

```
bin/kafka-configs.sh --bootstrap-server localhost:9093 --alter \
  --add-config 'SCRAM-SHA-512=[password=new-secret]' \
  --entity-type users --entity-name producer-user
```

**Real-Time Usage:** 7+ year audit-log retention for regulatory compliance; audit trails for incident forensics; automated 30–90 day credential rotation via Vault/CyberArk.

### 9.7 Complete Security Configuration Summary

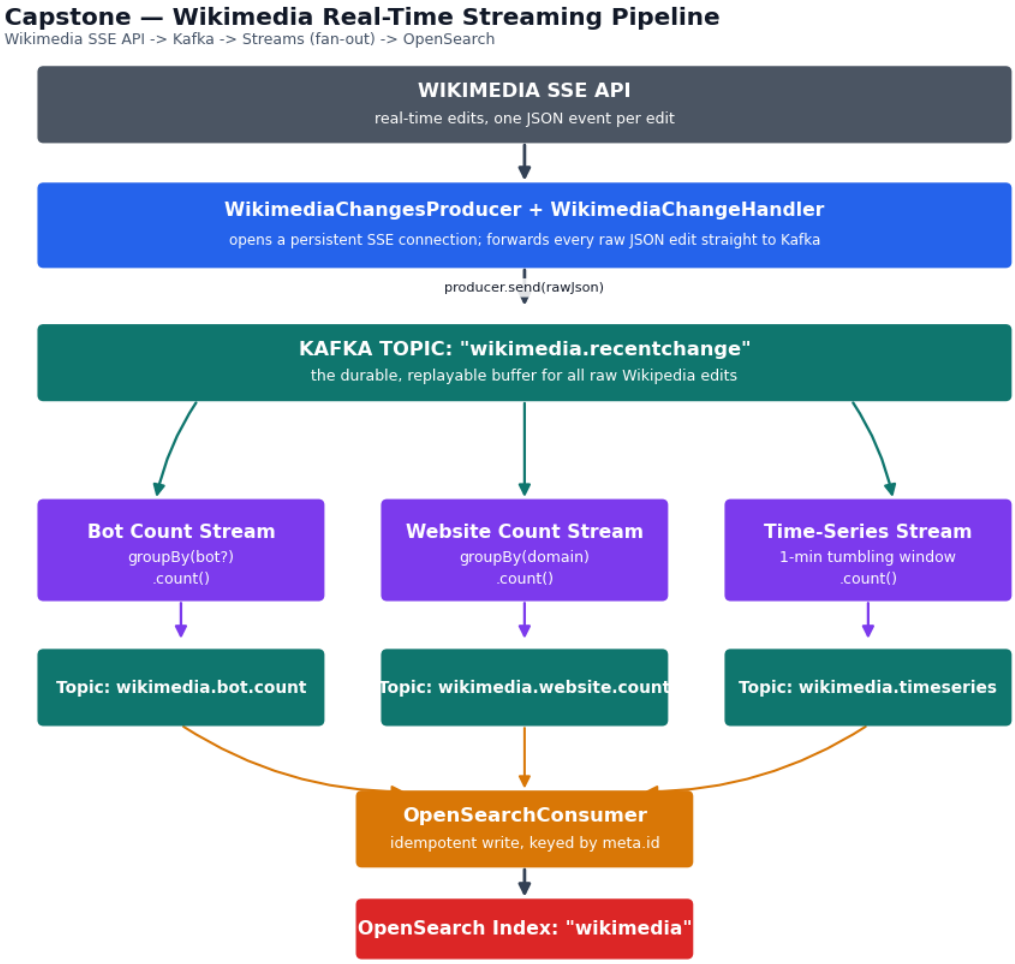
| Layer | Configuration | Example |
|---|---|---|
| Encryption (TLS) | `security.protocol=SSL` or `SASL_SSL` | `ssl.truststore.location`, `ssl.keystore.location` |
| Authentication (SASL) | `sasl.mechanism=SCRAM-SHA-512` | `sasl.jaas.config` with username/password |
| Authorization (ACLs) | `authorizer.class.name=AclAuthorizer` | `kafka-acls.sh --add --allow-principal` |
| Network Segmentation | Distinct `advertised.listeners` per protocol | `INTERNAL:PLAINTEXT,EXTERNAL:SASL_SSL` |
| Auditing | `log4j.logger.kafka.authorizer.logger=DEBUG` | Separate authorizer log file |
| Key Rotation | Scheduled cert/credential updates | Rolling broker restarts |

## Capstone Project — Wikimedia Real-Time Streaming Pipeline

A real-time pipeline that ingests live Wikipedia edits, processes them with Kafka Streams, and indexes them into OpenSearch for visualization.

```
wikimedia-streaming-pipeline/
├── pom.xml
├── docker-compose.yml
├── src/main/java/
│   ├── producer/
│   │   ├── WikimediaChangesProducer.java
│   │   └── WikimediaChangeHandler.java
│   ├── consumer/
│   │   └── OpenSearchConsumer.java
│   └── streams/
│       ├── WikimediaStreamsApp.java
│       ├── BotCountStreamBuilder.java
│       ├── WebsiteCountStreamBuilder.java
│       └── EventCountTimeseriesBuilder.java
└── src/main/resources/
    └── application.properties
```

## Architecture Diagram — End-to-End Pipeline




*Capstone - Wikimedia Real-Time Streaming Pipeline*

*Figure: Capstone - Wikimedia Real-Time Streaming Pipeline*

**What each stage means:**

- **Producer stage:** `WikimediaChangeHandler` implements the SSE `EventHandler` interface; every `onMessage` callback forwards the raw JSON straight to Kafka — no transformation here, so downstream consumers can decide how to interpret it.
- **Kafka topic stage:** `wikimedia.recentchange` acts as the durable, replayable buffer between the fast-moving Wikimedia firehose and the (potentially slower) downstream processors.
- **Streams stage:** three independent, stateless-to-stateful transformations all read the same topic in parallel (fan-out), each producing its own derived output topic — this is the Kafka Streams "multiple views of one log" pattern.

- **Consumer/sink stage:** `OpenSearchConsumer` performs the idempotent write into OpenSearch, keyed by the Wikimedia event's own `meta.id` so re-delivered records overwrite rather than duplicate.

**pom.xml**

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.wikimedia</groupId>
    <artifactId>wikimedia-streaming-pipeline</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <maven.compiler.source>11</maven.compiler.source>
        <maven.compiler.target>11</maven.compiler.target>
        <kafka.version>3.6.0</kafka.version>
        <opensearch.version>2.5.0</opensearch.version>
        <jackson.version>2.15.2</jackson.version>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.apache.kafka</groupId>
            <artifactId>kafka-clients</artifactId>
            <version>${kafka.version}</version>
        </dependency>
        <dependency>
            <groupId>org.apache.kafka</groupId>
            <artifactId>kafka-streams</artifactId>
            <version>${kafka.version}</version>
        </dependency>
        <dependency>
            <groupId>org.opensearch.client</groupId>
            <artifactId>opensearch-rest-high-level-client</artifactId>
            <version>${opensearch.version}</version>
        </dependency>
        <dependency>
            <groupId>com.fasterxml.jackson.core</groupId>
            <artifactId>jackson-databind</artifactId>
            <version>${jackson.version}</version>
        </dependency>
        <dependency>
            <groupId>com.fasterxml.jackson.datatype</groupId>
            <artifactId>jackson-datatype-jsr310</artifactId>
            <version>${jackson.version}</version>
        </dependency>
        <dependency>
```

```
            <groupId>com.launchdarkly</groupId>
            <artifactId>eventsource</artifactId>
            <version>2.1.0</version>
        </dependency>
        <dependency>
            <groupId>org.slf4j</groupId>
            <artifactId>slf4j-simple</artifactId>
            <version>2.0.9</version>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <version>3.11.0</version>
                <configuration>
                    <source>11</source>
                    <target>11</target>
                </configuration>
            </plugin>
        </plugins>
    </build>
</project>
```

**docker-compose.yml**

```
version: '3'
services:
  zookeeper:
    image: confluentinc/cp-zookeeper:latest
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
      ZOOKEEPER_TICK_TIME: 2000
    ports: ["2181:2181"]

  kafka:
    image: confluentinc/cp-kafka:latest
    depends_on: [zookeeper]
    ports: ["9092:9092"]
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
      KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 1
      KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 1

  opensearch:
    image: opensearchproject/opensearch:latest
```

```
    environment:
      discovery.type: single-node
      plugins.security.disabled: "true"   # dev-only; enable security
in production
    ports: ["9200:9200", "9600:9600"]
```

**application.properties**

```
bootstrap.servers=localhost:9092

application.id=wikimedia-streams-app
default.key.serde=org.apache.kafka.common.serialization.Serdes$StringS
erde
default.value.serde=org.apache.kafka.common.serialization.Serdes$Strin
gSerde

group.id=wikimedia-consumer-group
auto.offset.reset=earliest
enable.auto.commit=false

acks=all
retries=2147483647
enable.idempotence=true

opensearch.host=localhost
opensearch.port=9200
opensearch.scheme=http
```

### Producer Layer

**WikimediaChangeHandler.java:**

```
package producer;

import com.launchdarkly.eventsource.EventHandler;
import com.launchdarkly.eventsource.MessageEvent;
import org.apache.kafka.clients.producer.KafkaProducer;
import org.apache.kafka.clients.producer.ProducerRecord;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class WikimediaChangeHandler implements EventHandler {
    private static final Logger log =
LoggerFactory.getLogger(WikimediaChangeHandler.class);
    private final KafkaProducer<String, String> producer;
    private final String topic;

    public WikimediaChangeHandler(KafkaProducer<String, String>
producer, String topic) {
        this.producer = producer;
        this.topic = topic;
```

```
    }

    @Override public void onOpen() { log.info("SSE connection to
Wikimedia opened."); }
    @Override public void onClosed() { log.info("SSE connection to
Wikimedia closed."); }

    @Override
    public void onMessage(String event, MessageEvent messageEvent) {
        String data = messageEvent.getData();
        if (data == null || data.isEmpty()) return;

        producer.send(new ProducerRecord<>(topic, data), (metadata,
exception) -> {
            if (exception != null) {
                log.error("Failed to send message to Kafka: {}",
exception.getMessage());
            } else {
                log.debug("Sent to Kafka. Partition: {}, Offset: {}",
metadata.partition(), metadata.offset());
            }
        });
    }

    @Override public void onComment(String comment) { /* ignore SSE
comments */ }
    @Override public void onError(Throwable t) { log.error("Error in
SSE stream: {}", t.getMessage()); }
}
```

**WikimediaChangesProducer.java:**

```
package producer;

import com.launchdarkly.eventsource.EventSource;
import org.apache.kafka.clients.producer.KafkaProducer;
import org.apache.kafka.clients.producer.ProducerConfig;
import org.apache.kafka.common.serialization.StringSerializer;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import java.net.URI;
import java.util.Properties;

public class WikimediaChangesProducer {
    private static final Logger log =
LoggerFactory.getLogger(WikimediaChangesProducer.class);

    public static void main(String[] args) throws Exception {
        log.info("Starting Wikimedia Producer...");
```

```
        Properties props = new Properties();
        try (var input =
WikimediaChangesProducer.class.getClassLoader()
                .getResourceAsStream("application.properties")) {
            props.load(input);
        }

        props.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG,
StringSerializer.class.getName());
        props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG,
StringSerializer.class.getName());
        props.put(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, "true");
        props.put(ProducerConfig.ACKS_CONFIG, "all");

        String topic = "wikimedia.recentchange";

        try (KafkaProducer<String, String> producer = new
KafkaProducer<>(props)) {
            WikimediaChangeHandler handler = new
WikimediaChangeHandler(producer, topic);

            EventSource.Builder builder = new EventSource.Builder(
                    handler,
URI.create("https://stream.wikimedia.org/v2/stream/recentchange"));
            EventSource eventSource = builder.build();
            eventSource.start();

            log.info("Listening to Wikimedia stream. Press Ctrl+C to
stop.");
            Runtime.getRuntime().addShutdownHook(new Thread(() -> {
                log.info("Shutting down producer...");
                eventSource.close();
                producer.flush();
                producer.close();
            }));
            Thread.currentThread().join();
        }
    }
}
```

### OpenSearch Consumer Layer

**OpenSearchConsumer.java:**

```
package consumer;

import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import org.apache.http.HttpHost;
import org.apache.kafka.clients.consumer.ConsumerConfig;
```

```java
import org.apache.kafka.clients.consumer.ConsumerRecords;
import org.apache.kafka.clients.consumer.KafkaConsumer;
import org.apache.kafka.common.serialization.StringDeserializer;
import org.opensearch.action.index.IndexRequest;
import org.opensearch.action.index.IndexResponse;
import org.opensearch.client.RequestOptions;
import org.opensearch.client.RestClient;
import org.opensearch.client.RestHighLevelClient;
import org.opensearch.common.xcontent.XContentType;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import java.time.Duration;
import java.util.List;
import java.util.Properties;

public class OpenSearchConsumer {
    private static final Logger log =
LoggerFactory.getLogger(OpenSearchConsumer.class);
    private static final ObjectMapper mapper = new ObjectMapper();

    public static void main(String[] args) throws Exception {
        log.info("Starting OpenSearch Consumer...");

        Properties props = new Properties();
        try (var input = OpenSearchConsumer.class.getClassLoader()
                .getResourceAsStream("application.properties")) {
            props.load(input);
        }
        props.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG,
StringDeserializer.class.getName());
        props.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG,
StringDeserializer.class.getName());
        props.put(ConsumerConfig.ENABLE_AUTO_COMMIT_CONFIG, "false");
        props.put(ConsumerConfig.GROUP_ID_CONFIG, "opensearch-
consumer-group");

        try (RestHighLevelClient opensearchClient = new
RestHighLevelClient(
                RestClient.builder(new HttpHost("localhost", 9200,
"http")))) {

            try (KafkaConsumer<String, String> consumer = new
KafkaConsumer<>(props)) {
                consumer.subscribe(List.of("wikimedia.recentchange"));

                while (true) {
                    ConsumerRecords<String, String> records =
consumer.poll(Duration.ofMillis(1000));
                    if (records.isEmpty()) continue;
```

```
                    records.forEach(record -> {
                        try {
                            String json = record.value();
                            JsonNode jsonNode = mapper.readTree(json);
                            String id =
jsonNode.path("meta").path("id").asText();
                            if (id == null || id.isEmpty()) {
                                id = record.topic() + "_" +
record.partition() + "_" + record.offset();
                            }
                            IndexRequest request = new
IndexRequest("wikimedia")
                                    .id(id).source(json,
XContentType.JSON);
                            IndexResponse response =
opensearchClient.index(request, RequestOptions.DEFAULT);
                            log.debug("Indexed to OpenSearch: ID={},
Result={}", id, response.getResult());
                        } catch (Exception e) {
                            log.error("Failed to index record: {} |
Error: {}", record.value(), e.getMessage());
                        }
                    });

                    consumer.commitSync();
                    log.info("{} records processed and committed.",
records.count());
                }
            }
        }
    }
}
```

### Kafka Streams Layer

**WikimediaStreamsApp.java:**

```
package streams;

import org.apache.kafka.streams.KafkaStreams;
import org.apache.kafka.streams.StreamsBuilder;
import org.apache.kafka.streams.StreamsConfig;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import java.util.Properties;
import java.util.concurrent.CountDownLatch;

public class WikimediaStreamsApp {
    private static final Logger log =
LoggerFactory.getLogger(WikimediaStreamsApp.class);
```

```
    public static void main(String[] args) throws Exception {
        log.info("Starting Wikimedia Streams Application...");

        Properties props = new Properties();
        try (var input = WikimediaStreamsApp.class.getClassLoader()
                .getResourceAsStream("application.properties")) {
            props.load(input);
        }
        props.put(StreamsConfig.APPLICATION_ID_CONFIG,
props.getProperty("application.id"));
        props.put(StreamsConfig.BOOTSTRAP_SERVERS_CONFIG,
props.getProperty("bootstrap.servers"));
        props.put(StreamsConfig.DEFAULT_KEY_SERDE_CLASS_CONFIG,

"org.apache.kafka.common.serialization.Serdes$StringSerde");
        props.put(StreamsConfig.DEFAULT_VALUE_SERDE_CLASS_CONFIG,

"org.apache.kafka.common.serialization.Serdes$StringSerde");

        StreamsBuilder builder = new StreamsBuilder();
        new BotCountStreamBuilder().buildPipeline(builder);
        new WebsiteCountStreamBuilder().buildPipeline(builder);
        new EventCountTimeseriesBuilder().buildPipeline(builder);

        KafkaStreams streams = new KafkaStreams(builder.build(),
props);

        CountDownLatch latch = new CountDownLatch(1);
        Runtime.getRuntime().addShutdownHook(new Thread(() -> {
            log.info("Shutting down streams...");
            streams.close();
            latch.countDown();
        }));

        streams.start();
        log.info("Kafka Streams started successfully!");
        latch.await();
    }
}
```

**BotCountStreamBuilder.java:**

```
package streams;

import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import org.apache.kafka.common.serialization.Serde;
import org.apache.kafka.common.serialization.Serdes;
import org.apache.kafka.streams.StreamsBuilder;
```

```
import org.apache.kafka.streams.kstream.Consumed;
import org.apache.kafka.streams.kstream.Grouped;
import org.apache.kafka.streams.kstream.KStream;
import org.apache.kafka.streams.kstream.Produced;

public class BotCountStreamBuilder {
    private static final ObjectMapper mapper = new ObjectMapper();

    public void buildPipeline(StreamsBuilder builder) {
        Serde<String> stringSerde = Serdes.String();
        KStream<String, String> sourceStream = builder.stream(
                "wikimedia.recentchange", Consumed.with(stringSerde,
stringSerde));

        KStream<String, String> botFlagStream =
sourceStream.mapValues(value -> {
            try {
                JsonNode json = mapper.readTree(value);
                boolean isBot = json.path("bot").asBoolean(false);
                return isBot ? "bot" : "human";
            } catch (Exception e) {
                return "unknown";
            }
        });

        botFlagStream
                .groupByKey(Grouped.with(stringSerde, stringSerde))
                .count()
                .toStream()
                .to("wikimedia.bot.count", Produced.with(stringSerde,
Serdes.Long()));
    }
}
```

**WebsiteCountStreamBuilder.java:**

```
package streams;

import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import org.apache.kafka.common.serialization.Serde;
import org.apache.kafka.common.serialization.Serdes;
import org.apache.kafka.streams.StreamsBuilder;
import org.apache.kafka.streams.kstream.Consumed;
import org.apache.kafka.streams.kstream.Grouped;
import org.apache.kafka.streams.kstream.KStream;
import org.apache.kafka.streams.kstream.Produced;

public class WebsiteCountStreamBuilder {
    private static final ObjectMapper mapper = new ObjectMapper();
```

```
    public void buildPipeline(StreamsBuilder builder) {
        Serde<String> stringSerde = Serdes.String();
        KStream<String, String> sourceStream = builder.stream(
                "wikimedia.recentchange", Consumed.with(stringSerde,
stringSerde));

        KStream<String, String> websiteStream =
sourceStream.mapValues(value -> {
            try {
                JsonNode json = mapper.readTree(value);
                String serverName = json.path("server_name").asText();
                if (serverName == null || serverName.isEmpty()) {
                    serverName =
json.path("meta").path("domain").asText();
                }
                return serverName != null ? serverName : "unknown";
            } catch (Exception e) {
                return "unknown";
            }
        });

        websiteStream
                .groupByKey(Grouped.with(stringSerde, stringSerde))
                .count()
                .toStream()
                .to("wikimedia.website.count",
Produced.with(stringSerde, Serdes.Long()));
    }
}
```

**EventCountTimeseriesBuilder.java:**

```
package streams;

import org.apache.kafka.common.serialization.Serde;
import org.apache.kafka.common.serialization.Serdes;
import org.apache.kafka.streams.StreamsBuilder;
import org.apache.kafka.streams.kstream.Consumed;
import org.apache.kafka.streams.kstream.Grouped;
import org.apache.kafka.streams.kstream.KStream;
import org.apache.kafka.streams.kstream.Materialized;
import org.apache.kafka.streams.kstream.Produced;
import org.apache.kafka.streams.kstream.TimeWindows;
import org.apache.kafka.streams.kstream.Windowed;
import java.time.Duration;

public class EventCountTimeseriesBuilder {

    public void buildPipeline(StreamsBuilder builder) {
```

```
        Serde<String> stringSerde = Serdes.String();
        Serde<Long> longSerde = Serdes.Long();

        KStream<String, String> sourceStream = builder.stream(
                "wikimedia.recentchange", Consumed.with(stringSerde,
stringSerde));

        sourceStream
                .mapValues(value -> "all") // static key -> global 1-
minute aggregation
                .groupByKey(Grouped.with(stringSerde, stringSerde))
                .windowedBy(TimeWindows.ofSizeWithNoGrace(Duration.ofM
inutes(1)))
                .count(Materialized.with(stringSerde, longSerde))
                .toStream()
                .map((Windowed<String> key, Long count) -> {
                    String windowStart =
key.window().startTime().toString();
                    return new
org.apache.kafka.streams.KeyValue<>(windowStart, count.toString());
                })
                .to("wikimedia.timeseries", Produced.with(stringSerde,
stringSerde));
    }
}
```

### Running the End-to-End Pipeline

```
# 1. Start infrastructure
docker-compose up -d

# 2. Create the input topic
docker exec -it wikimedia-streaming-pipeline-kafka-1 kafka-topics.sh \
  --create --topic wikimedia.recentchange \
  --bootstrap-server localhost:9092 \
  --partitions 3 --replication-factor 1

# 3. Run the Kafka Streams app
mvn compile exec:java -Dexec.mainClass="streams.WikimediaStreamsApp"

# 4. Run the OpenSearch consumer
mvn compile exec:java -Dexec.mainClass="consumer.OpenSearchConsumer"

# 5. Run the Wikimedia producer (ingests live data)
mvn compile exec:java -
Dexec.mainClass="producer.WikimediaChangesProducer"
```

### Validating the Pipeline

```
# Check OpenSearch index
curl -X GET "http://localhost:9200/wikimedia/_search?pretty" \
  -H "Content-Type: application/json" \
```

```
  -d '{"query":{"match_all":{}},"size":1}'

# Bot counts
docker exec -it wikimedia-streaming-pipeline-kafka-1 kafka-console-
consumer.sh \
  --topic wikimedia.bot.count --bootstrap-server localhost:9092 --
from-beginning

# Website counts
docker exec -it wikimedia-streaming-pipeline-kafka-1 kafka-console-
consumer.sh \
  --topic wikimedia.website.count --bootstrap-server localhost:9092 --
from-beginning

# Time-series (every minute)
docker exec -it wikimedia-streaming-pipeline-kafka-1 kafka-console-
consumer.sh \
  --topic wikimedia.timeseries --bootstrap-server localhost:9092 --
from-beginning
```

---

## Appendix — Master CLI Cheat Sheet

### Topics

```
bin/kafka-topics.sh --create --topic <t> --bootstrap-server
localhost:9092 --partitions 3 --replication-factor 1
bin/kafka-topics.sh --list --bootstrap-server localhost:9092
bin/kafka-topics.sh --describe --topic <t> --bootstrap-server
localhost:9092
bin/kafka-topics.sh --alter --topic <t> --bootstrap-server
localhost:9092 --partitions 6
```

### Producing / Consuming

```
bin/kafka-console-producer.sh --topic <t> --bootstrap-server
localhost:9092
bin/kafka-console-producer.sh --topic <t> --bootstrap-server
localhost:9092 --property parse.key=true --property key.separator=:
bin/kafka-console-consumer.sh --topic <t> --bootstrap-server
localhost:9092 --from-beginning
bin/kafka-console-consumer.sh --topic <t> --bootstrap-server
localhost:9092 --partition 0 --offset 2
```

### Offsets

```
bin/kafka-run-class.sh kafka.tools.GetOffsetShell --topic <t> --
bootstrap-server localhost:9092
bin/kafka-consumer-groups.sh --describe --group <g> --bootstrap-server
localhost:9092
bin/kafka-consumer-groups.sh --group <g> --bootstrap-server
localhost:9092 --to-earliest --execute
```

```
bin/kafka-consumer-groups.sh --group <g> --bootstrap-server
localhost:9092 --to-latest --execute
bin/kafka-consumer-groups.sh --group <g> --bootstrap-server
localhost:9092 --to-offset 50 --topic <t>:0 --execute
bin/kafka-consumer-groups.sh --group <g> --bootstrap-server
localhost:9092 --to-datetime 2026-07-12T12:00:00.000 --execute
```

### Consumer Groups

```
bin/kafka-consumer-groups.sh --list --bootstrap-server localhost:9092
bin/kafka-consumer-groups.sh --group <g> --bootstrap-server
localhost:9092 --delete
```

### Cluster / KRaft

```
docker exec -it kafka1 kafka-metadata-quorum.sh --bootstrap-server
localhost:9092 describe --status
docker exec -it kafka1 kafka-leader-election.sh --bootstrap-server
localhost:9092 --topic <t> --election-type PREFERRED --partitions
0,1,2
```

### Security (ACLs)

```
bin/kafka-acls.sh --bootstrap-server localhost:9093 --add --allow-
principal User:<u> --operation Write --topic <t>
bin/kafka-acls.sh --bootstrap-server localhost:9093 --list
bin/kafka-configs.sh --bootstrap-server localhost:9093 --alter --add-
config 'SCRAM-SHA-512=[password=<p>]' --entity-type users --entity-
name <u>
```

---

*End of document. This guide preserves every definition, diagram, code sample, and CLI command developed across Chapters 1–9 and the Wikimedia capstone project, reorganized into a single reference with a master architecture overview and professionally rendered, annotated flow diagrams for quick onboarding.*